

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

ROBOTAKSİ – BİNEK OTONOM ARAÇ YARIŞMASI

KRİTİK TASARIM RAPORU

(HAZIR ARAÇ KATEGORİSİ)

TAKIM ADI: RACLAB Sigun

TAKIM ID: 398527

İçindekiler

1.	Takım Organizasyonu	3
1.1.	Organizasyon Şeması ve Ekip Üyeleri	3
1.2.	İş Zaman Çizelgesi	5
2.	Ön Tasarım Raporu Değerlendirmesi	6
3.	Hazır Araç Özellikleri ve Analizi	7
3.1.	Sensörler	8
3.1.1.	Velodyne Puck Hi-Res LIDAR	8
3.1.2.	ZED 2 Stereo Kamera	9
3.1.3.	SBG Ellipse microll IMU	10
4.	Araç Kontrol Ünitesi	10
4.1.	Araç Bilgisayarı	10
4.2.	Haberleşme	12
5.	Araç Dinamiği Modelleme/Tanımlama ve Kontrolü	12
5.1.	Aracın Matematiksel ve Deneysel Modeli	12
5.2.	Aracın Teknik Özellikleri	14
6.	Otonom Sürüş Algoritmaları	14
6.1.	Levha ve Işık Tanıma	14
6.2.	Şerit Tespit Algoritması	16
6.3.	Yer Analiz Algoritması	17
6.4.	Engelden Kaçınma Algoritması	18
6.5.	Derin Öğrenme ile Şerit Tespiti	19
6.6.	Park Algoritması	20
6.7.	Optimum Tekerlek Açısı Algoritması	21
6.8.	Sinyal Algoritması	22
7.	Güvenlik Önlemleri	22
7.1.	SOLID İlkeleri	22
7.2.	LIDAR Sensörü ile Alınan Güvenlik Önlemleri	23
7.3.	Şeritten Çıkma Durumu	23
7.4.	Nesne Tespiti Önlemleri	23
7.5.	Araç Hareket Halindeyken Gelen Sensör Verilerinin Uzun Süreli Kesilmesi	24
7.6.	Araç Üzerinde Bulunan Güvenlik Önlemleri	24
8.	Simülasyon	24
8.1.	Yarışma Senaryosu İçin Simülasyon Sonuçları	25
9.	Sistem Entegrasyonu	27
9.1.	ROS Mimarisi	28
9.2.	Nesne Tespiti Mimarisi	28
9.3.	CAN Bus Haberleşme Kurulumu	28
9.4.	ZED Kamera Entegrasyonu	29
9.5.	LIDAR Sensörü Entegrasyonu	29
9.6.	IMU ve GPS Sensör Entegrasyonları	29
10.	Test ve Doğrulama	29
11.	Referanslar	33

1. Takım Organizasyonu

RACLAB topluluğu 2014 tarihinde Doç. Dr. Akif DURDU tarafından Konya Teknik Üniversitesi'nde kurulmuştur. RACLAB topluluğu bünyesindeki takımlar, otonom teknolojiler alanında faaliyet göstermekte ve alt dal olarak İnsansız Sualtı Aracı, İnsansız Kara Aracı (İKA) ve İnsansız Hava Aracı (İHA) kategorilerinde çalışmaktadır.

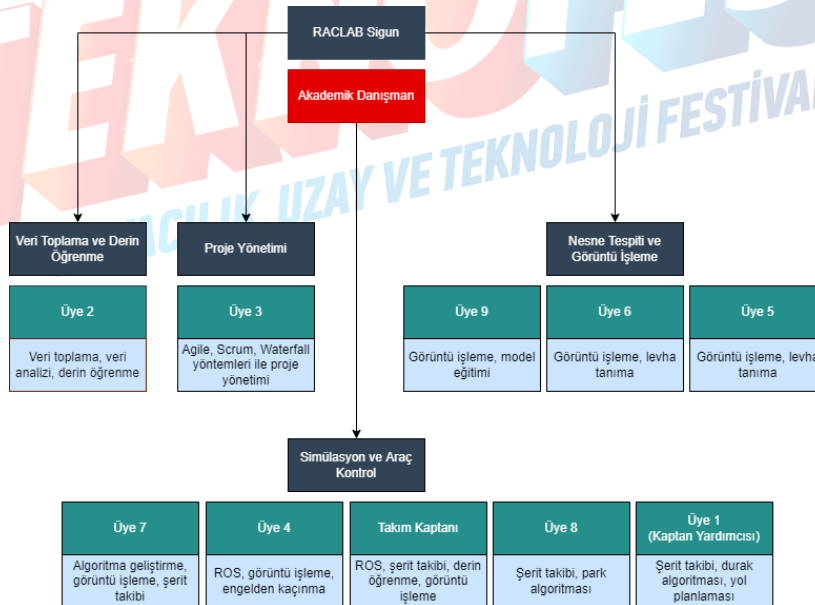
RACLAB Sigun takımı, İnsansız Kara Aracı alanında ulusal ve uluslararası yarışmalarda faaliyet göstermektedir. 2019 yılında RACLAB takımı, Bilkent Teknopark'ta bulunan Amerika'nın en prestijli üniversitelerinden biri olan MIT (Massachusetts Institute Of Technology) öncülüğünde OpenZeka firmasının düzenlediği otonom araç yarışmasında birincilik elde etmiştir.

RACLAB Sigun takımı, 1 doktora, 9 lisans öğrencisi olmak üzere 10 kişiden oluşmaktadır. Otonom araç takımı, Konya Teknik Üniversitesi Elektrik Elektronik Mühendisliği Bölümü Öğretim Üyesi Doç. Dr. Akif DURDU danışmanlığında, 2 Elektrik Elektronik Mühendisliği, 1 Endüstri Mühendisliği ve 7 Bilgisayar Mühendisliği öğrencisiyle beraber çalışmalarına devam etmektedir. Otonom araç takımı proje kapsamında 4 alt gruba ayrılmıştır. Bu gruplar veri toplama ve derin öğrenme, nesne tespiti ve görüntü işleme, proje yönetimi simülasyon ve araç kontrol ekibidir. RACLAB Sigun takımı organizasyon şeması Şekil 1'de verilmiştir.

Takımımız, çalışmalarını yürütürken RACLAB topluluğu bünyesinde bulunan lisansüstü ve doktora öğrencilerinin güncel teknolojiler üzerine yürüttükleri projelerden de destek alarak çalışmalarını ilerletmektedir. Hedefleri doğrultusunda yazılım teknolojileri alanında tecrübe edinerek gelecekte bu alanda yerli mühendisler yetiştirme vizyonu ile hareket etmektedir.

Yarışma kapsamında yapılan çalışmalar ilk olarak Gazebo ortamında gerçekleştirilmiştir. Gazebo ortamında başarılı olan çalışmalar, ikinci aşamada 1/10 ölçekli RACLAB mini aracı üzerinde test edilmiştir. İkinci aşamada testi geçen çalışmalar, yarışma aracı üzerinde test edilmiş ve bulunan tüm hataların giderilmesi sağlanmıştır.

1.1. Organizasyon Şeması ve Ekip Üyeleri



Şekil 1: Takım organizasyonu

Takım üyeleri hakkında genel tanıtıcı kısa bilgiler sırasıyla aşağıda belirtilmiştir.

Takım Kaptanı: Sakarya Üniversitesi Bilgisayar Programcılığı bölümü mezunudur. Konya Teknik Üniversitesi Bilgisayar Mühendisliği bölümü 3. sınıf öğrencisidir. Yapay zekâ, insansız kara araçları ve otonom sürüş algoritmaları gibi alanlara ilgi duymaktadır. Takımda şerit takibi, otonom sürüş algoritmaları ve görüntü işleme alanlarında çalışmaktadır.

Üye 1: Giresun Üniversitesi Bilgisayar Programcılığı bölümü mezunudur. Konya Teknik Üniversitesi Bilgisayar Mühendisliği bölümü 3. Sınıf öğrencisidir. Yapay zekâ, derin öğrenme ve otonom sistemler gibi alanlara ilgi duymaktadır. Takımda şerit takibi ve otonom sürüş algoritmaları alanlarında çalışmaktadır.

Üye 2: Kabil Üniversitesi Bilgisayar Bilimleri bölümü mezunudur. Yüksek lisansını Osmania Üniversitesi Bilgisayar Bilimi Mühendisliği bölümünde tamamlamış olup, doktora eğitimine Konya Teknik Üniversitesi Bilgisayar Mühendisliği bölümünde devam etmektedir. Araştırma alanları arasında otonom sistemlerin genel alanı, yerelleştirme, sensör füzyonları ve olasılıklı durum tahmin modellerinin yanı sıra karar verme, otonom navigasyon, SLAM uygulamaları yer almaktadır. Takımda görüntü işleme ve veri analizi üzerinde çalışmalar yapmaktadır.

Üye 3: Konya Teknik Üniversitesi Endüstri Mühendisliği bölümü 1. sınıf öğrencisidir. Agile (Çevik) proje yönetimi, Waterfall (Şelale) metodolojisi gibi alanlara ilgi duymaktadır. Takım içerisinde takımın tanıtılması, sponsorluk anlaşmalarının yapılması ve takımın organizasyonunun sağlanması konularında çalışmaktadır.

Üye 4: Konya Teknik Üniversitesi, Mühendislik ve Doğa Bilimleri Fakültesi, Elektrik-Elektronik Mühendisliği bölümü 3.sınıf öğrencisidir. Yapay zekâ, insansız hava araçları, insansız kara araçları ve sürü İHA sistemleri gibi alanlara ilgi duymaktadır. Takımda ROS tabanlı algoritmalar alanında çalışmaktadır.

Üye 5: Kırıkkale Üniversitesi Bilgisayar Programcılığı bölümü mezunudur. Konya Teknik Üniversitesi Bilgisayar Mühendisliği bölümü 3. sınıf öğrencisidir. Siber güvenlik, yapay zekâ, görüntü işleme, veri analizi ve veri bilimi gibi alanlara ilgi duymaktadır. Takımda görüntü işleme ve nesne tespiti alanlarında çalışmaktadır.

Üye 6: Sakarya Üniversitesi Bilgisayar Programcılığı bölümü mezunudur. Konya Teknik Üniversitesi Bilgisayar Mühendisliği 3. sınıf öğrencisidir. İnsansız kara araçları, yapay zekâ, siber güvenlik gibi alanlara ilgi duymaktadır. Takımda görüntü işleme, görüntü optimizasyonu ve simülasyon üzerinde harita oluşturulması üzerine çalışmaktadır.

Üye 7: Konya Teknik Üniversitesi, Mühendislik ve Doğa Bilimleri Fakültesi, Elektrik-Elektronik Mühendisliği bölümü 4.sınıf öğrencisidir. Yapay zekâ, görüntü işleme ve otonom sürüş algoritmaları gibi alanlara ilgi duymaktadır. Takımda görüntü işleme ve algoritma geliştirme alanlarında çalışmaktadır.

Üye 8: Konya Teknik Üniversitesi Bilgisayar Mühendisliği bölümü 3. sınıf öğrencisidir. Robotik, web ve yapay zekâ gibi alanlarına ilgi duymaktadır. Takımda otonom sürüş algoritmaları ve haritalama alanlarında çalışmaktadır.

Üye 9: Konya Teknik Üniversitesi Bilgisayar Mühendisliği bölümü 3. sınıf öğrencisidir. Yapay zekâ, veri bilimi, robotik ve görüntü işleme gibi alanlara ilgi duymaktadır. Takımda görüntü işleme alanında çalışmaktadır.

1.2. İş Zaman Çizelgesi

Tablo 1: Çalışma çizelgesi¹

Çalışmalar	Aylar																	
	Kasım		Aralık		Ocak		Şubat		Mart		Nisan		Mayıs		Haziran		Temmuz	
Literatür taraması ve TEKNOFEST yarışma kurallarının incelenmesi	✓	✓	✓	✓														
Otonom sürüş için gerekli ortamların öğrenilmesi		✓	✓															
Simülasyon ortamının kurulması ve yarışma ortamına uygun harita tasarlanması		✓	✓	✓	✓	✓	✓											
Lokalizasyon ve haritalandırma algoritmalarının tasarlanması		✓	✓	✓	✓													
Levha tespiti algoritmalarının tasarlanması		✓	✓	✓	✓													
Şerit tespit ve takip algoritmalarının tasarlanması		✓	✓	✓	✓													
Derin öğrenme için veri toplanması	✓	✓	✓	✓	✓													
Trafik işareti ve levha algılama algoritmalarının tasarlanması				✓	✓	✓												
Levha tespiti ve şerit takibiyle beraber durak algoritmalarının tasarlanması					✓	✓	✓	✓	✓	✓								
Park etme algoritmalarının tasarlanması			✓	✓	✓	✓	✓	✓	✓	✓								
Otonom sürüş algoritmalarının tasarlanması				✓	✓	✓	✓											
Tasarlanan algoritmaların simülasyon ortamında test edilmesi				✓	✓	✓	✓	✓										
Simülasyon ortamında test edilen algoritmaların mini araçta test edilmesi								✓	✓	✓	✓							
Mini araçta test edilen algoritmaların gerçek araca entegre edilmesi											✓	✓	✓	✓	✓	✓		
Hata tespiti ve iyileştirme çalışmaları													✓	✓	✓	✓	✓	

¹ Çalışma çizelgesinde aylar 2 parçaya bölünerek planlama yapılmıştır.

Tablo 1’de verilen iş paketleri arasından önem arz eden ana iş paketlerinin gereksinimleri ve hedefleri kısaca şunlardır:

Literatür taraması ve TEKNOFEST yarışma kurallarının incelenmesi: Bu pakette yarışma şartnamesinin incelenmesiyle şartnameye uygun literatür taraması yapılmakta ve başarılı bir sonuç elde edilmesi hedeflenmektedir.

Simülasyon ortamının kurulması ve yarışma ortamına uygun harita tasarlanması: TEKNOFEST tarafından yayınlanan yarışma şartnamesine uygun bir simülasyon ortamının seçilmesi, yarışmaya uygun bir parkurun oluşturulması ve oluşturulan parkurun simülasyon ortamına aktarılması hedeflenmektedir.

Şerit tespit ve takip algoritmalarının tasarlanması: Otonom sürüş için gerekli algoritmalar arasında birinci sırada olan şerit tespit ve takip algoritmalarının geliştirilmesi ve geliştirilen algoritmaların doğru bir şekilde çalışması hedeflenmektedir.

Otonom sürüş algoritmalarının tasarlanması: Şerit tespit ve takip algoritmalarıyla bağlantılı olan diğer otonom sürüş algoritmaların geliştirilmesi ve geliştirilen algoritmaların doğru bir şekilde sonuç vermesi hedeflenmektedir.

Hata tespiti ve iyileştirme çalışmaları: Bu pakette tasarım ve geliştirme aşamalarında karşılaşılabilecek muhtemel hataların tespit edilmesi ve bulunan hataların giderilmesi, ayrıca geliştiren algoritmaların iyileştirilmesi hedeflenmektedir.

2. Ön Tasarım Raporu Değerlendirmesi

Ön tasarım raporu aşamasında Gazebo simülasyon ortamında çalışmalar yapılmıştır. Çalışmalar yapılırken simülasyon tarafında bulunan hatalar giderilmiştir.

Ön tasarım raporu sürecinin ardından şerit değiştirme için çalışmalar yapılmıştır. Bu çalışmalar kapsamında, IMU sensöründen gelen verilerin kullanılması kararlaştırılmıştır. IMU sensöründen gelen veriler, hesaplamalara dahil edildiği için otonom sürüş algoritmalarında değişiklikler yapılmıştır.

Parkurdaki levhaların tespiti için hem simülasyon ortamında hem de gerçek ortamlarda çalışmalar yapılmıştır. Şartnamede verilen trafik levhaları temel olarak alınmış olup TTVS (Türkiye Trafik Veri Seti) kullanılarak bir veri seti oluşturulmuştur [1]. Elde edilen veri seti kullanarak, derin öğrenme algoritmaları yardımıyla çeşitli nesne tespit modelleri oluşturulmuştur. Yapılan testlerde YOLOv5 algoritmasının ön plana çıktığı gözlemlenmiştir. YOLOv5 algoritması üzerinde çalışmalar yapılmış olup optimum tespit seviyesine ulaşılmıştır. Hazırlanan veri seti, YOLOv5 üzerinde eğitilmiş ve sonuç modeli Bilişim Vadisi’ndeki test sürecinde denenmiştir. Test sürecinde modelin hatalı tespit yaptığı durumlar incelenmiş ve bu hataların çözümü için çeşitli veriler toplanmış olup yeni veriler veri setine eklenmiştir.

Ön tasarım raporu sürecinde 13500 veri etiketlenerek oluşturulan veri setiyle model eğitimi gerçekleştirilmiştir. Kritik tasarım raporu aşamasında 40500 veriyle beraber veri seti üç katına çıkarılmış olup yeni sınıflar eklenerek sınıf ve veri sayısı arttırılmıştır. Oluşturulan yeni veri setiyle model eğitimi gerçekleştirilmiş olup modelin başarı oranı kayda değer miktarda artmıştır.

Simülasyon ortamında ROS mimarisi kullanılarak düğümlerle araç hareketleri ve haberleşme sağlanmaktaydı. Kritik tasarım raporu sürecinde ise hazır araç üzerinde ve simülasyon ortamında ROS mimarisinden yararlanmanın yanı sıra CAN Bus sistemiyle beraber çalışacak şekilde algoritmalarda düzenlemeler yapılmıştır.

3. Hazır Araç Özellikleri ve Analizi

TEKNOFEST tarafından, yarışmada kullanılması amacıyla bir hazır araç platformu sunulmuştur. Hazır araç platformu, Tragger T-Car modelini temel almaktadır. Elektrikli bir araç olan T-Car, Ottomotive firmasının hazırlamış olduğu araç kontrol ünitesi ile otonomlaştırılmıştır. Araç üzerinde Velodyne Puck Hi-Res LIDAR, ZED 2 Camera, SBG Ellipse microII IMU, Ottomotive Araç Kontrol Ünitesi bulunmaktadır. Araç, haberleşme için CAN protokolünü kullanmaktadır. CAN protokolünde gerekli haberleşmeyi sağlamak için çalışmalar yapılmıştır. Sensörler arası haberleşmeyi sağlamak için gerekli paketler araç kontrol ünitesinde bulunacaktır. Tragger T-Car model aracın teknik özellikleri tablolarda verilmiştir.

Tablo 2: Tragger T-Car Teknik Verileri

Motor	7.4 KW Schaubmuller
Tork / Nm (Max)	125 NM
Çalışma Gerilimi (V)	48 V
Batarya	Trojan T145 Plus
Oturma Kapasitesi	2, 4, 6, 8
Max. Araç Hızı (km/h)	24
Şasi	6000 Serisi Uçak Sınıfı Alüminyum

Tablo 3: Tragger T-Car Süspansiyon Sistemi

Ön Süspansiyon	Bağımsız Teleskopik Amortisör
Arka Süspansiyon	Helezon Yaylı Sabit Aks / Teleskopik Amortisör
Lastikler	10" Duro

Tablo 4: Tragger T-Car Fren Sistemi

Park Freni	Mekanik / Ayak Pedallı
Servis Freni	Hidrolik / Kampana

Tablo 5: Tragger T-Car Araç Parçaları ve Modelleri

Oriental Motor	BLVM640N-GFS_ORIENTAL
Manyetik Kavrama	ABK 04 3.1 DIZAYN
Lineer Aktüatör	MD24A050-0100CNO2NNSD
Enkoder	8.F5888.5B2F.2123
LIDAR	Velodyne Puck Hi-Res 16
IMU	SBG Ellipse microII
Kamera	ZED 2 Stereo
Otonom Kontrolör	NVIDIA Jetson AGX Xavier
Araç Kontrolörü	Ottomotive Araç Kontrol Ünitesi 3x Can Bus 1x GPS 1x RS485 1x GSM

Tablo 6: Tragger T-Car Fiziksel Özellikleri

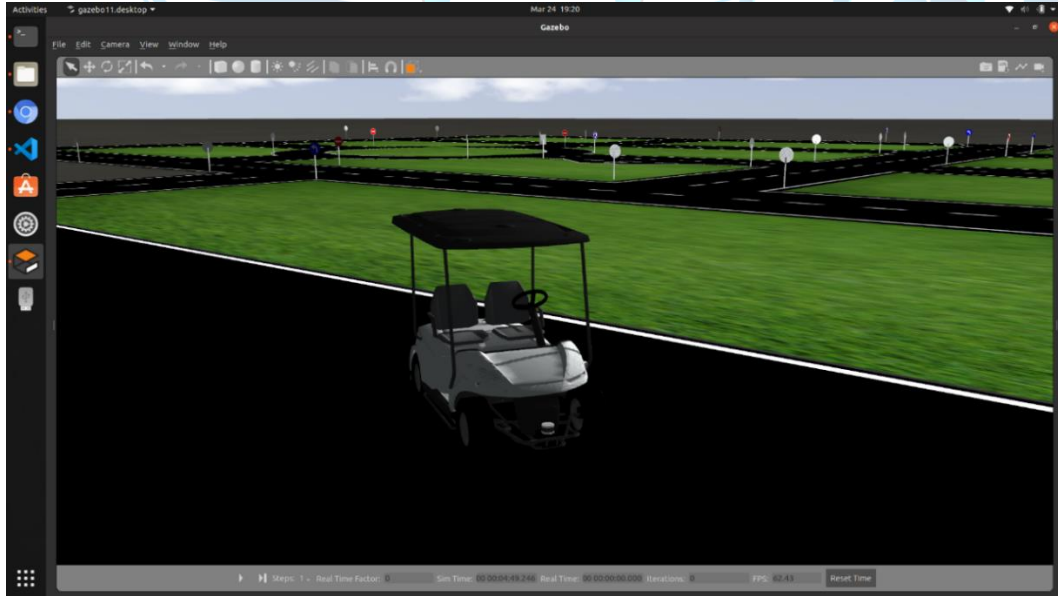
Dingil Mesafesi	2400
Toplam Uzunluk	2385
Ön Uzunluk	290
Arka Uzunluk	595
Şasi Yerden Yüksekliği (Kanopsisiz)	185
Toplam Yükseklik (Kanopsisiz)	1240
Toplam Yükseklik (Kanopili)	1950
Toplam Genişlik (Kanopsisiz)	1330
Arka İz Genişliği	1115
Ön İz Genişliği	1030

3.1. Sensörler

Araç üzerinde Velodyne Puck Hi-Res LIDAR, ZED 2 Stereo Kamera, SBG Ellipse microII IMU sensörleri bulunmaktadır. Alt başlıklarda bu sensörler hakkında detaylı bilgiler verilmiştir.

3.1.1. Velodyne Puck Hi-Res LIDAR

Velodyne Puck Hi-Res LIDAR, belirli bir menzil içerisindeki değişken mesafeleri ölçmek için darbeli lazer şeklinde ışığı kullanan bir uzaktan algılama sensörüdür [2]. Bu sensör 360° gerçek zamanlı veriyi üç boyutlu uzaklıklar ve kalibre edilmiş yansımalar olarak sisteme kaydetmektedir. Sistem tarafından kaydedilen diğer verilerle birleştirilen ışık darbeleri menzil içindeki engeller hakkında yüksek doğrulukta üç boyutlu veriler üretmektedir. LIDAR sensörü yardımıyla engel tespiti ve engelden kaçış algoritmalarının kullanılması mümkün olmaktadır. Simülasyon ortamında bulunan araç üzerindeki LIDAR sensör görünümü Şekil 2’de gösterilmiştir. Velodyne Puck Hi-Res LIDAR sensörünün teknik verileri Tablo 7’de verilmiştir.



Şekil 2: Gazebo Simülasyon ortamında LIDAR görünümü

Tablo 7: Velodyne Puck Hi-Res LIDAR Teknik Özellikleri

Kanal Sayısı	16 Kanal
Ölçüm Mesafesi	100 m
Hassasiyet	3 cm
Görüş Açısı	360 °
Point Sayısı	~600.000
Bağlantı Türü	100 Mbps Ethernet
Ağırlık	~830 g
Çalışma Sıcaklığı	-10 °C - 60 °C

3.1.2. ZED 2 Stereo Kamera

ZED 2 stereo kamera, Stereolabs şirketi tarafından üretilen insan görüş sistemini temel alarak geliştirilmiş bir kameradır. ZED 2 stereo kamera üzerinde iki adet kamera bulunmaktadır. Bu iki kamera yardımıyla derinlik verisi oluşturulmakta ve derinlik verisi yardımıyla piksellerin konumu belirlenebilmektedir. Düşük ışık hassasiyetine sahip olan ZED 2 stereo kamera iç ve dış ortamlarda 2K çözünürlüğe kadar video çekebilmektedir. ZED 2 stereo kamera, 0,5 ile 20 metre arasında 32 bitlik derinlik verisi oluşturmaktadır. ZED 2 stereo kamera, sanal odometri özelliği ile 1mm hareket ve 0,1 derece dönme hassasiyeti ile 6 ekseninde hareketleri algılayabilmektedir. ZED 2 stereo kamera genellikle arttırılmış gerçeklik, 3 boyutlu haritalama, drone, otonom araç ve analizler için 3 boyutlu veri toplama uygulamalarında kullanılmaktadır. Windows ve Ubuntu işletim sistemlerinde çalışabilen ZED 2 stereo kameranın, ROS (Robot Operating System), Unity, Matlab, OpenCV gibi platformlara uygun paketleri bulunmaktadır [3]. Simülasyon ortamındaki araçta bulunan ZED kameranın görüş açısı Şekil 3'te verilmiştir. ZED stereo kameranın teknik özellikleri Tablo 8'de verilmiştir.



Şekil 3: Gazebo simülasyon ortamında ZED kamera görüşü

Tablo 8: ZED 2 Stereo Kamera Teknik Özellikleri

Çıkış Çözünürlüğü	Yan yana 2x (2208x1242 15fps) 2x (1920x1080 30fps) 2x (1280x720 60fps) 2x (672x376 100fps)
Görüş Açısı	110°(H) x 70°(V) x 120°(D)
Bağlantı	USB 2.0/3.0
Derinlik Mesafesi	0.3 m - 20 m
Ağırlık	166g
Çalışma Sıcaklığı	-10°C ile +45°C arasında
Boyut	175 x 30 x 33 mm
Güç Tüketimi	5V USB / 380mA

3.1.3. SBG Ellipse microll IMU

Inertial Measurement Unit (IMU) olarak da bilinen Ataletsel Ölçüm Birimi, hızlanmayı, yönü, açısal oranları ve diğer yerçekimsel kuvvetleri ölçer ve bildirir. Üç ivmeölçer, üç jiroskop ve yön gereksinimine göre üç manyetometreden oluşur [4]. Bu üç donanımın her biri bir eksenini temsil eder. Bu eksenler sırasıyla boylamsal, yatay ve dikey eksenlerdir. Çeşitli IMU türleri mevcuttur. Fiber optik jiroskop (FOG) teknolojiye sahip IMU'lar, halka lazer jiroskop (RLG) teknolojiye sahip IMU'lar, mikro elektro-mekanik sistemler (MEMS) teknolojiye sahip IMU'lar olarak sınıflandırılabilir. Tekerlekten alınan odometri verisiyle IMU verileri füzyon edilerek CAN protokolü aracılığıyla daha doğru bir veri elde edilir. Bu verilerle birlikte aracın doğru şekilde dönüp dönmediği kontrol edilebilmektedir. SBG Ellipse microII IMU sensörünün teknik özellikleri Tablo 9'da verilmiştir.

Tablo 9: SBG Ellipse microII IMU Teknik Özellikleri

	İvmeölçer	Jiroskop	Manyetometre
Mesafe	16 g	450 °/s	50 Gauss
Hizalama Hatası	<0,05 °	<0,05 °	<0,1 °
Band Genişliği	390 Hz	133 Hz	22 Hz

4. Araç Kontrol Ünitesi

Bu kısımda, Ottomotive firması tarafından geliştirilip hazır araca entegre edilen araç kontrol ünitesi içerisinde bulunan cihazlar hakkında detaylı bilgiler verilmiştir.

4.1. Araç Bilgisayarı

Araç bilgisayarı olarak NVIDIA Jetson AGX Xavier geliştirici kartı kullanılmıştır. Bu kart gömülü yapay zekâ özellikli bir bilgi işlem cihazı olup yüksek hız ve güç verimliliği sağlamaktadır. Modüler mimariye sahip bu bilgisayar; 512-core Volta GPU (Tensor Çekirdekleriyle), 8-core ARM v8, 2 CPU, 32 GB'a kadar bellek, 137 GB/s bellek bant genişliğiyle uçtan uca AI ve robotik uygulamalarının geliştirilmesine olanak sağlamaktadır [5]. Otonom sürüş algoritmalarının bilgisayar üzerinde kurulması ve yapay sinir ağının kullanımının araç üzerinde çalıştırılacak olmasından dolayı oldukça güçlü ve gerçek zamanlı yüksek FPS ihtiyacı olduğu için bilgisayar seçimi otonom teknolojilerde oldukça önemlidir. CUDA çekirdek yazılımı ile senkron bir şekilde birden fazla modelle kullanabileceğimiz Jetson'u farklı senaryolarda yüksek performans ya da enerji tasarrufu ayarı ile kullanılabilmektedir. NVIDIA, bunun için NVPModel'i sunmaktadır. Çalışmalarda kullanılan ROS tabanlı yazılmış görüntü işleme ve nesne tespiti algoritmalarının yüksek işlem gücüne sahip bir araç bilgisayarıyla kullanılması sayesinde algoritmaların çalışma performanslarını olumlu yönde etkilemektedir. NVIDIA Jetson AGX Xavier geliştirme kartının görünümü Şekil

4'te verilmiştir. NVIDIA Jetson AGX Xavier geliştirme kartının teknik özellikleri Tablo 10'da verilmiştir.

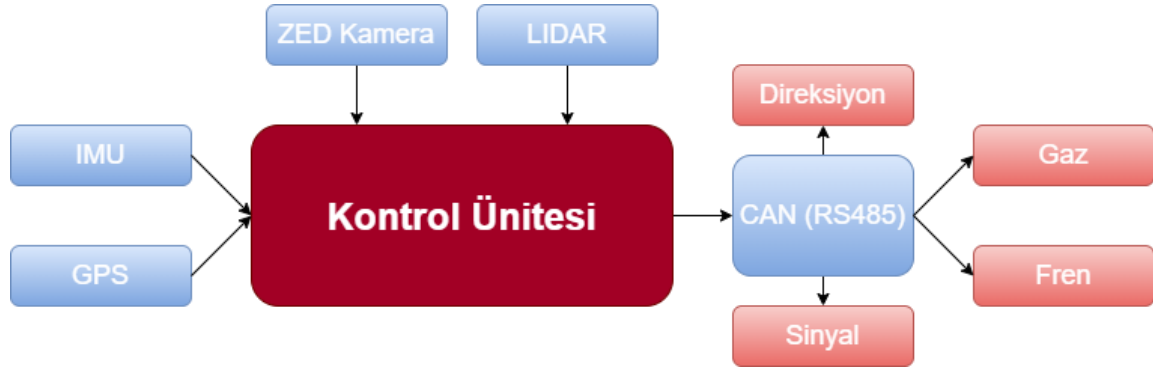


Şekil 4: NVIDIA Jetson AGX Xavier Geliştirici Kartı

Tablo 10: NVIDIA Jetson AGX Xavier Geliştirici Kartı Teknik Özellikleri

CPU	8 çekirdekli NVIDIA Carmel Armv8.2 64 bit CPU 8 MB L2 + 4 MB L3
GPU	512 adet CUDA çekirdekli 64 adet tensor çekirdekli NVIDIA Volta mimarisi
Yapay Zeka Performansı	32 TOPS
DL Hızlandırıcı	2 adet NVDLA
Görüş Hızlandırıcı	2 adet PVA
Bellek	32 GB 256 bit LPDDR4x 136,5 GB/sn
Depolama	32 GB eMMC 5.1
Güç	10 W 15 W 30 W

Araç kontrol ünitesinde bulunan sensörlerin ve araç hareket sisteminin modellemesi çıkarılmış ve çalışmalar bu modellemeye uygun şekilde yürütülmüştür. Çıkarılan modelleme Şekil 5'te verilmiştir.



Şekil 5: Araç Kontrol Ünitesi Modellemesi

4.2. Haberleşme

Araçlarda haberleşme protokolü olarak CAN (Controller Area Network), UART ve Flexray gibi protokoller kullanılmaktadır. CAN protokolünü diğer protokollerden ayıran özellikler haberleşmede kullanılan cihazlardan birisi bozulduğunda iletişim ağının bozulmaması ve iletişim esnasında elektriksel gürültülerden dolayı oluşabilecek hatalardan etkilenmeyecek bir altyapıya sahip olmasıdır. Bu sebepten dolayı kullanılan sensörlerin ana bilgisayar olmadan birbirleriyle araç içerisinde haberleşmeleri için CAN Bus kullanılmaktadır [6]. Bu veri yolunu kullanarak araç içerisinde yer alan sensörler ve aygıtlar arası iletişim ana bilgisayar olmadan birbirleriyle CAN Bus ile sağlanacaktır. İletişim sonucunda oluşan veriler işlenerek araç bilgisayarına gönderilecektir. Geliştirilen otonom sürüş algoritmalarında yer alan hız, fren, sinyal ve tekerlek açısı gibi araç komutları CAN Bus üzerinden mikro denetleyicilere ve sensörlere iletilecektir [7].

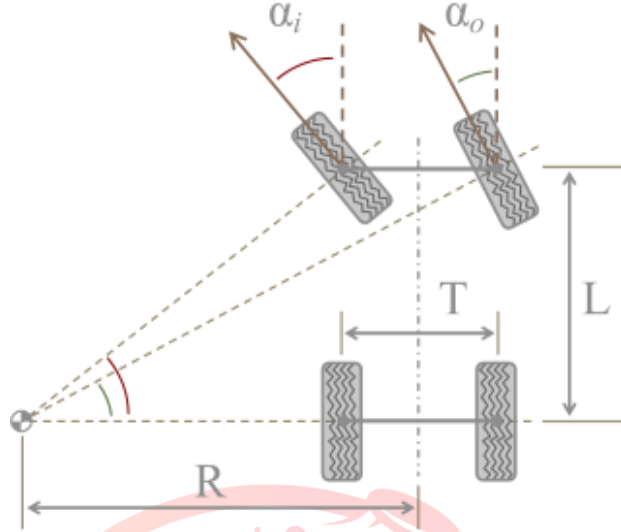
5. Araç Dinamiği Modelleme/Tanımlama ve Kontrolü

Bu kısımda, TEKNOFEST tarafından sağlanan hazır aracın dinamiği, kontrolü ve teknik bilgileri hakkında detaylı bilgiler verilmiştir.

5.1. Aracın Matematiksel ve Deneysel Modeli

Araç modelinin hareketi için kullanılan yaklaşım Ackermann yönlendirme geometrisidir. Bu geometri genel olarak tekerlek üzerinde statik olarak tanımlı olan “Toe” açısının dinamik olarak değişmesini sağlar. Bu açı, araca üstten bakıldığında hareketi sağlayan ön tekerleklerin arkaya göre farklı mesafede olması durumudur. Ön tarafın arkaya göre kapalı olmasına toe-in, açık olmasına toe-out denir [8]. Ackermann yönlendirme geometrisi sayesinde direksiyon döndürüldüğünde hangi tekerleğin kaç derecelik bir açı yapması gerektiğini hesaplamamızı sağlamaktadır. Böylelikle her bir ön tekerleğin birbirinden bağımsız bir şekilde dönmesi sayesinde aracın dönüşte kayması önlenir.

Düzlemsel hareket yapan aracın hareketi esnasında her tekerleği bir yay üzerinde hareket etmektedir. Dönüş yaparken taşıtın kaymasını engellemek, aracın dönüş kararlılığını korumak ve minimum tekerlek aşınmasını sağlamak gerekir. Bunun için de tekerleklerin taradıkları yayların merkezinin yani ani dönme merkezlerinin çakışık olması koşulundan geçer. Bu duruma Ackermann geometrisi adı verilir [9]. Ackermann geometrisindeki her tekerleğin merkez çizgi arasındaki mesafe (T), aracın dingil mesafesi (L), tekerleğin dümdüz ileriye olan açısı (α_1), dış tekerleğin düz önden açısı (α_0) ve dönüş yarıçapı (R)’e göre formüle edilebilir [10]. Formüle edilmiş olan görüntü Şekil 6’da verilmiştir.

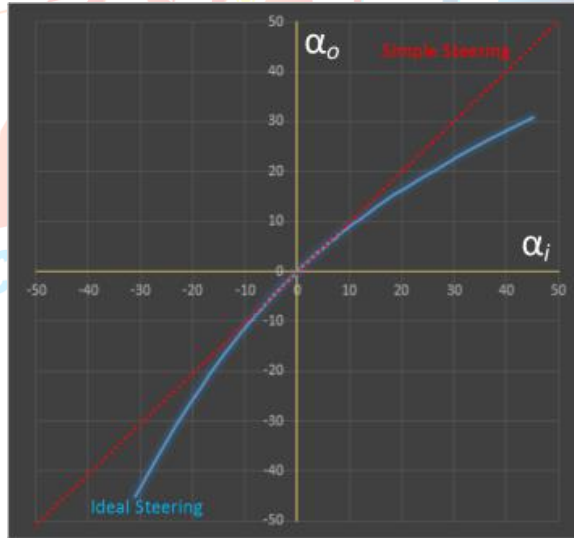


Şekil 6: Ackermann Geometrisi'nin Üstten Görünümü

Sabit bir hızda yalnızca ön tekerleklerin yönlendirildiğini varsayarsak, tekerleklerde ideal açılar bulmak için aşağıdaki formül kullanılır.

$$\alpha_i = \tan^{-1} \left(\frac{L}{R - \frac{T}{2}} \right) \quad \alpha_o = \tan^{-1} \left(\frac{L}{R + \frac{T}{2}} \right) \quad (1)$$

Şekil 7'de verilen grafikte aracın dingil mesafesi (L) ve tekerleğin merkez çizgi arasındaki mesafe (T) sabit tutularak, x ekseninde iç tekerlek açısı ve y ekseninde olası bir eğri aralığına karşılık gelen dış tekerlek açısı çizilmiştir.



Şekil 7: Ackermann Dönüş Grafiği

Kırmızı çizgi, basit dönüş için iki tekerlek açısı arasındaki ilişkiyi gösterir. Mavi çizgi, ideal dönüş için iki tekerlek açısı arasındaki ilişkiyi gösterir. Dönüş ne kadar sıkı olursa, iç açının dış açıya oranı o kadar yüksek olur. Tekerlekleri bu şekilde döndürmek için bir mekanizmanın nasıl oluşturulacağı konusundaki zorluk, Ackermann yönlendirme geometrisi sayesinde çözülmüştür.

5.2. Aracın Teknik Özellikleri

Aracın teknik özellikleri Tablo 11’de verilmiştir.

Tablo 11: Aracın Teknik Özellikleri

Motor	7.4 KW Schaubmuller
Tork / Nm (Max)	125 NM
Çalışma Gerilimi (V)	48 V
Batarya	Trojan T145 Plus
Oturma Kapasitesi	2, 4, 6, 8
Max. Araç Hızı (km/h)	24
Şasi	6000 Serisi Uçak Sınıfı Alüminyum
Ön Süspansiyon	Bağımsız Teleskopik Amortisör
Arka Süspansiyon	Helezon Yaylı Sabit Aks / Teleskopik Amortisör
Lastikler	10” Duro
Park Freni	Mekanik / Ayak Pedallı
Servis Freni	Hidrolik / Kampana
Oriental Motor	BLVM640N-GFS_ORIENTAL
Manyetik Kavrama	ABK 04 3.1 DIZAYN
Lineer Aktüatör	MD24A050-0100CNO2NNSD

6. Otonom Sürüş Algoritmaları

Aracın belirtilen görevleri yerine getirmesi ve sorunsuz, güvenli bir şekilde otonom sürüş gerçekleştirmesi için geliştirilmiş olan otonom sürüş algoritmalarından bu kısımda bahsedilmiştir.

6.1. Levha ve Işık Tanıma

Trafik levhaları için literatür araştırması yapılmıştır. Karayolları Genel Müdürlüğü’nün yayınlamış olduğu Trafik İşaretleri El Kitabı temel alınarak parkurda mevcut olabilecek levhaların analizi yapılmıştır [11]. Levha tespiti için uygun olabilecek nesne tespit algoritmalarının bir listesi oluşturulmuştur. Liste hazırlanırken aşağıdaki konular dikkate alınmıştır:

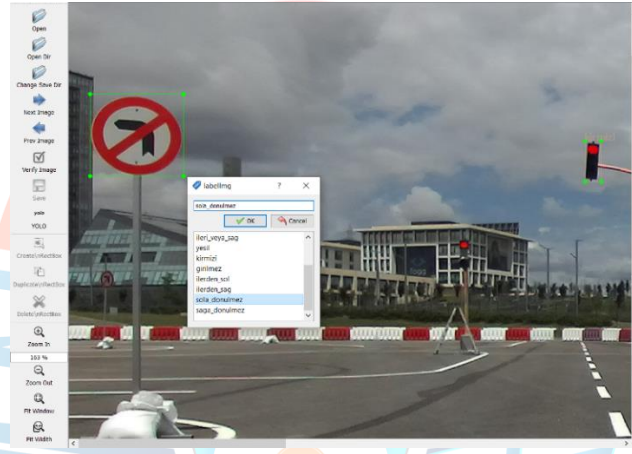
- Nesne tespit algoritmasının belirli bir veri seti üzerindeki doğru tespit oranı,
- Nesne tespit algoritmasının tespit hızı,
- Nesne tespit algoritmasındaki parametre sayısı,
- Nesne tespit algoritmasının kaynak kullanımı

Yapılan liste doğrultusunda araçta kullanılacak olan nesne tespit algoritmasının yüksek hızda çalışmasının ve doğru tespit oranının yüksek olması kararlaştırılmıştır. Listedeki çıktılar incelendiğinde, trafik levhası tespiti için en uygun algoritmanın YOLOv5 algoritması olduğu belirlenmiştir. YOLOv5 algoritması içerisinde çeşitli modeller mevcuttur. Bu modellerin belirli bir veri seti üzerindeki doğru tespit oranları ve tespit hızları incelenmiştir. İnceleme sonucunda YOLOv5 Small modelinin kullanılması kararlaştırılmıştır [12]. Çeşitli YOLOv5 modellerinin karşılaştırılması Şekil 8’de verilmiştir.

Model	size (pixels)	mAp ^{val} 0.5:0.95	mAp ^{val} 0.5	Speed CPU b1 (ms)	Speed V100 b1 (ms)	Speed V100 b32 (ms)	params (M)	FLOPs @640 (B)
YOLOv5n	640	28.0	45.7	45	6.3	0.6	1.9	4.5
YOLOv5s	640	37.4	56.8	98	6.4	0.9	7.2	16.5
YOLOv5m	640	45.4	64.1	224	8.2	1.7	21.2	49.0
YOLOv5l	640	49.0	67.3	430	10.1	2.7	46.5	109.1
YOLOv5x	640	50.7	68.9	766	12.1	4.8	86.7	205.7

Şekil 8: YOLOv5 Model Kıyaslaması [13]

Gerçek ortamlardan trafik levhaları için veri toplanmıştır. Veri toplanırken levhaların etiket sayısı eşit tutulmaya çalışılmıştır. Etiketleme işlemine yönelik kullanım, labelImg ortamında Şekil 9’da verilmiştir.



Şekil 9: Levha Tespiti

Trafik ışıklarının nesne tespit modelinde tanınabilmesi için ayrıca veri toplanmıştır. Trafik lambasının anlık olarak hangi renk ışığın yandığını tespit etme görevinde üç farklı sınıf kullanıldığında oluşan algoritmik karışıklıktan dolayı modelin verdiği oran düşük olmaktadır. Bu oranı yükseltmek için üç farklı sınıf yerine sadece trafik lambasını içeren tek bir sınıf oluşturulmuş ve trafik ışığının sırayla yanık olduğu kısımlardan eşit şekilde veri toplanarak derin öğrenme algoritması için etiketleme işlemi gerçekleştirilmiştir. Derin öğrenme algoritması için hazırlanan veriler ile veri seti oluşturularak eğitim işlemi gerçekleştirilmiştir. Derin öğrenme algoritmasının yanı sıra araç kamerasından alınan görüntünün tamamında görüntü işleme metodu çalıştırılmış ve bu işlem gerçekleştirilirken araç bilgisayarında kaynak kullanımının arttığı gözlemlenmiştir. Kaynak kullanımının azaltılması için görüntüde yer alan kullanılmayan kısımlarda kırpma işlemi uygulanmıştır. Böylelikle hem kaynak kullanımı azaltılmış hem de tespit edilen nesne içerisindeki görüntünün eşik değerleri hesaplanarak hangi renk ışığın yandığı algılanmıştır. Trafik lambası tespitine yönelik model ve algoritma çıktıları sırasıyla Şekil 10 (a) ve Şekil 10 (b)’de gösterilmiştir.



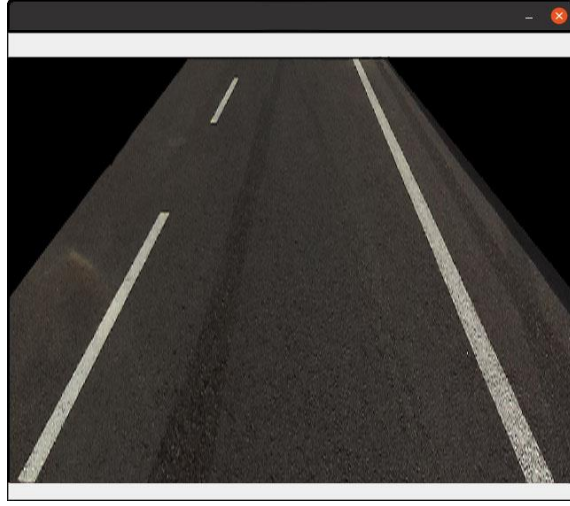
Şekil 10 (a): Gerçek Ortamda Trafik Lambası Model Çıktısı



Şekil 10 (b): Algoritma çıktısı

6.2. Şerit Tespit Algoritması

Şerit tespiti algoritmasında araç kamerasından gelen görüntü, sadece sağ ve sol şeridi görecektir. Yan kısımlardan kırılarak aracın sağ ve sol kenarlarında oluşabilecek olası nesne karmaşıklıkları engellenmiştir. Bu sayede kullanılmayan alanlar işlenmeyerek araç bilgisayarının fazla işlem yapıp kaynak kullanımını arttırmasının önüne geçilmiştir. Düzenlemeler sonucunda oluşan görüntü üzerinde OpenCV kütüphanesi kullanılarak görüntü işleme teknikleri uygulanmıştır. Kenarlarından kırılmış görüntü perspective transform yöntemi uygulanarak kuş bakışı görünüme getirilmiştir. Görüntü kuş bakışına dönüştürerek şeritlerin tespiti kolaylaştırılmıştır. Elde edilen kuş bakışı görünümü Şekil 11’de gösterilmiştir.



Şekil 11: Elde Edilen Kuş Bakışı Görünümü

Kuş bakışına çevrilen görüntü önce gri formata dönüştürülüp kanal sayısı azaltılarak daha kolay işlem yapılabilir hale getirilmiştir. Daha sonra gereksiz özelliklerden kurtulmak için görüntüye blur eklenmiştir. Blur eklenen görüntü üzerinde şerit çizgilerini ortaya çıkarmak ve görüntüdeki gürültüyü gidermek adına threshold yöntemi uygulanmıştır. Threshold uygulanmış görüntü Şekil 12’de gösterilmiştir.



Şekil 12: Threshold Uygulanmış Görüntü

Bahsedilen görüntü işleme metotları araç kamerasından alınan görüntü üzerinde kullanıldığında sağ ve sol şeritlerin tespiti sağlanmıştır. Tespit edilen şeritler kameranın sağ ve sol alt kısımlarındaki beyaz şerit konumlarına göre işlenerek şeritlerin araca olan uzaklıkları hesaplanmıştır. Elde edilen uzaklık verilerinin ortalaması alınarak, aracın şerit ortasında kalması sağlanmakta olup araç hareketini buna bağlı olarak gerçekleştirmektedir. Araç, şerit değişikliği yapacağı zaman IMU sensöründen gelen açisal hızı ve doğrusal ivme verisini işleyerek şerit değişikliği yapılmaktadır.

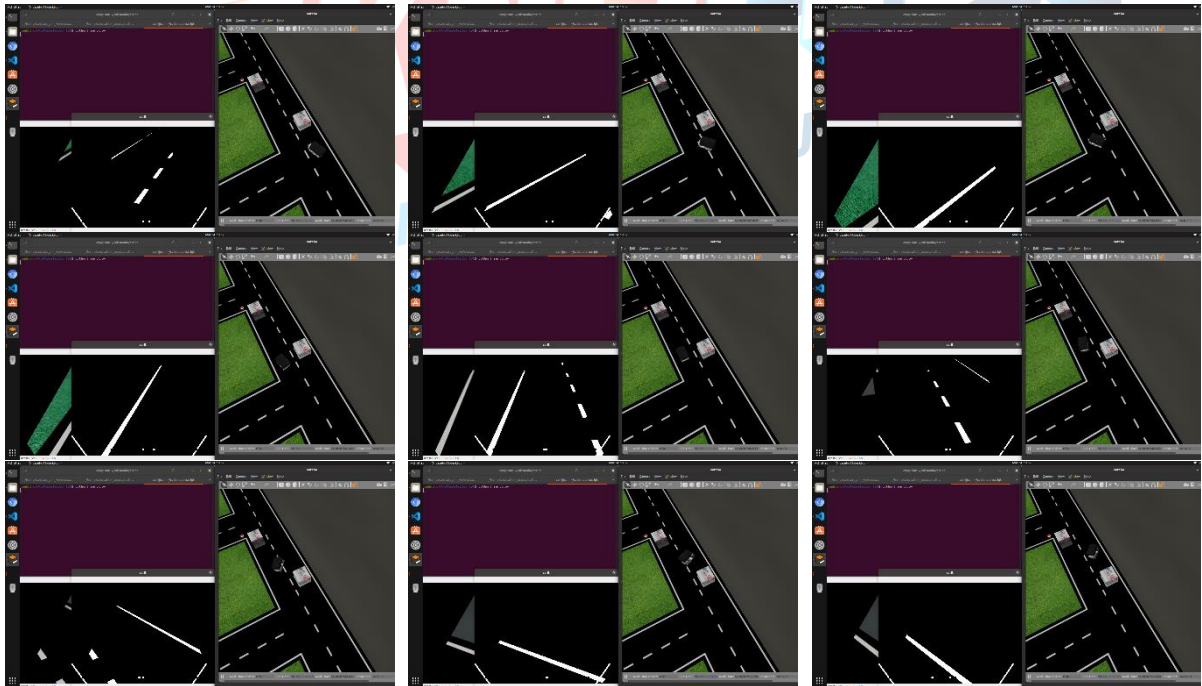
6.3. Yer Analiz Algoritması

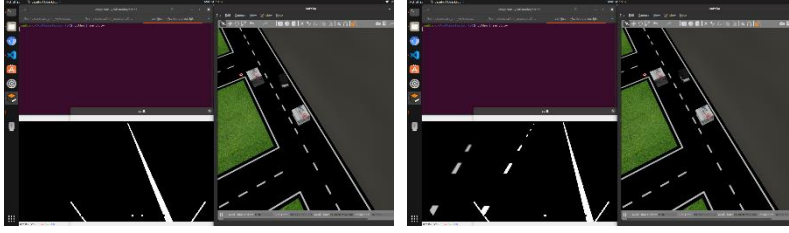
Otonom sürüş gerçekleştirilirken kaynak kullanımının azaltılması ve verimliliğin artırılması için yer analiz algoritması tasarlanmıştır. Bu algoritma sayesinde araç hareket halindeyken şeritlerin bulunduğu zeminin renk analizi yapılmaktadır. Renk analizi yapılırken yerin renk ortalaması alınmaktadır. Yapılan renk analiziyle birlikte parlaklığın çok olması durumunda

güneşin aracın bulunduğu noktaya etki yaptığı gözlemlenmiş ve güneş olduğu durumlarda gölge oluşturabileceği düşünülerek derin öğrenme ağırlıklı şerit takibi gerçekleştirilmiştir. Bu oran %70 oranında derin öğrenme modeliyle şerit tespiti, %30 oranında görüntü işleme metoduyla şerit tespiti şeklindedir. Yer analizinde parlaklığın az olması durumunda gölge oluşabilme ihtimalinin azalması göz önünde bulundurularak verimliliği arttırmak için derin öğrenme modelinin kullanımı azaltılmış olup %80 görüntü işleme metodu ve kritik noktalarda kullanılmak şartıyla %20 oranında derin öğrenme modeli kullanımıyla şerit tespiti ve şerit takibi yapılmıştır. Algoritma sayesinde derin öğrenme modeli ve görüntü işleme metotları arasında ağırlık değerleri verilerek ölçeklendirme yapılmış olup kaynak kullanımında azalma sağlanarak verimlilik artırılmıştır.

6.4. Engelden Kaçınma Algoritması

Araç seyir halindeyken sürülebilir alan içerisinde bir engel tespit edilmesi durumunda, araç yol üzerinde bulunan iki şeridin boş olma durumunu LIDAR yardımıyla kontrol etmektedir. Bu süreçte 360 derece veri üreten LIDAR'ın 180 derecelik ön kısmı 5 bölgeye ayrılmıştır. Bu bölgeler sol, ön sol, ön, ön sağ ve sağdır. Bu bölgeler baz alınarak engelden kaçınma algoritması uygulanmaktadır. Sağ şeritte hareket halinde ise araç ve bulunduğu şeritte engel algılanması durumunda sol şeridi LIDAR ile kontrol etmektedir. Bu alan kontrol edilirken aracın hızı, aracın bulunduğu konum, LIDAR'dan gelen ön sol bölge ve sol bölge değerleri baz alındığında sol şerit müsait durumda ise önündeki engelden kaçınarak sol şeride geçiş işlemi gerçekleşmektedir. Sağında kalan engel ile oluşan çarpışma risk durumunun ortadan kalkması halinde tekrardan sağ şeride geçiş yaparak planlanmış rotada hareketine devam etmektedir. Aynı şekilde sol şeritte hareket halinde iken araç önünde bir engel çıkması durumunda sağ şeridin müsait olma durumu LIDAR ile kontrol edilerek şerit değiştirme işlemi gerçekleştirilir. Engelle olan mesafe belirli bir seviyeye çıktıktan sonra tekrardan araç eski şeridine geçiş yapmaktadır. Önünde engelin bulunduğu ve diğer şeridinde dolu olduğu durumda araç hızı yavaşlatılarak durdurulur. Sürülebilir alanda yol müsaitliği oluşması halinde araç rotasını güncelleyerek hareketine devam etmektedir. Engelden kaçınma algoritmasını gösterir LIDAR görünümü ve kaçınma adımları Şekil 13'te verilmiştir.





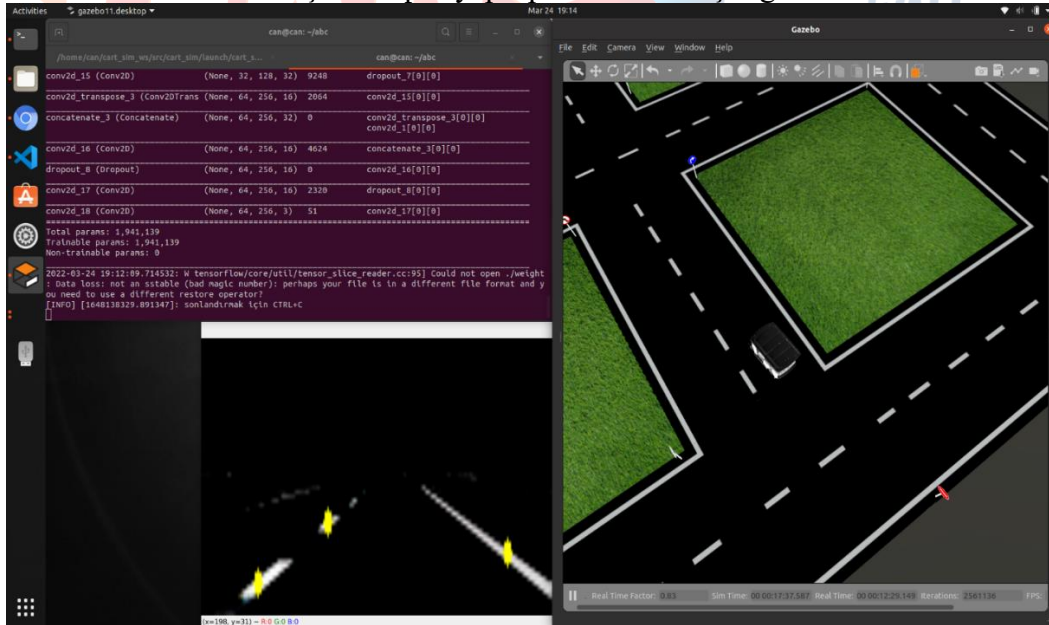
Şekil 13: Engelden Kaçınma Algoritmasının Adımları

6.5. Derin Öğrenme ile Şerit Tespiti

Derin öğrenme tabanlı şerit tespit algoritmasında ilk olarak eğitim için gerekli görseller oluşturulmuştur. Veri oluşturmak için hazırlanan simülasyon ortamındaki şeritlerde hareket eden araç kamerasıyla çekilen video görselleri kullanılmıştır. Kamera görüntüsüyle alınan şeritleri beyaz renge boyanarak veriler oluşturulmuştur. Derin öğrenme için giriş verisi [64,256,3] boyutunda kamera görüntüsü ve çıkış görüntüsü aynı boyutlarda siyah renk üzerine beyaz renge boyanmış şeritler olacak şekilde eğitim yapılmıştır.

Şerit tespiti uygulamasının en büyük sorunlarından birisi olan ışığa karşı duyarlılık sorununu çözebilmek ve eğitim verisini iyileştirmek amacıyla Data Augmentation(Veri çoğaltma) işlemi yapılmıştır. Oluşturulan veriler ilk olarak gamma düzeltmesi kullanılarak parlak ve karanlık olacak şekilde 3 katına çıkartılmıştır. U-net yapısı kullanılarak yapılan eğitim sonucunda oluşturulan modelin simülasyon ortamında gerçek zamanlı testleri yapılmıştır.

Tespit edilen şeritler üzerinde noktalar belirlenerek bu noktalara göre aracın kontrolü için gerekli tekerlek açısı değerleri hesaplanmıştır. Tespit edilen şeritleri, bulunduğu görüntünün alt kısmından başlayarak 6 parça ve [10,256] piksel büyüklüğünde resimler alınmıştır. Alınan bu 6 resim üzerinde beyaz renkler belirlenmiştir. Belirlenen renkleri içine alan en büyük çember belirlenip orta noktasının y eksenini koordinatı bir diziye kaydedilmiştir. Alınan 6 resim üzerinde de bu işlemler belirlenmiştir ve her renkten 6 tane olmak üzere toplam 18 nokta belirlenmiştir. Belirlenen noktalar Şekil 14'te gösterilmektedir. Araç normal sürüş sırasında belirlenen noktaları ortalamak hareket etmesi için tespit edilen noktaların ortalaması alınıp görselin orta noktasından farkına göre hata hesaplanmıştır. İşlenen son üç görselde oluşan hata elde edilerek tekerlek açısı değeri hesaplanmıştır. Hesaplama işleminde gerçekleşen sonuç neticesinde derin öğrenme modeli kullanılarak şerit tespiti yapıp otonom sürüş sağlanmaktadır.



Şekil 14: Şeritler ve Belirlenen Noktaların Görüntüsü

6.6. Park Algoritması

Park algoritması, üç farklı verinin birleşimiyle çalışmaktadır. Bu veriler şunlardır:

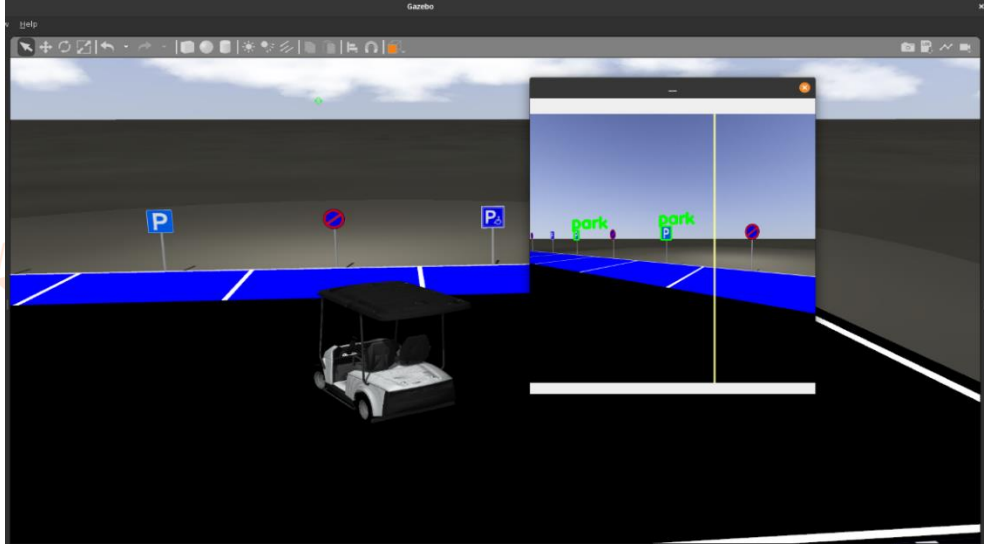
- Park tabelalarının köşe noktaları ve bu köşe noktalarından hesaplanan tabela orta noktası
- Aracın anlık yaw(yönelim) açısı
- Tabelaya olan uzaklık

Tabela orta noktası kullanılarak önceden belirlenen yönelim katsayısının çarpımıyla anlık hedef elde edilmektedir.

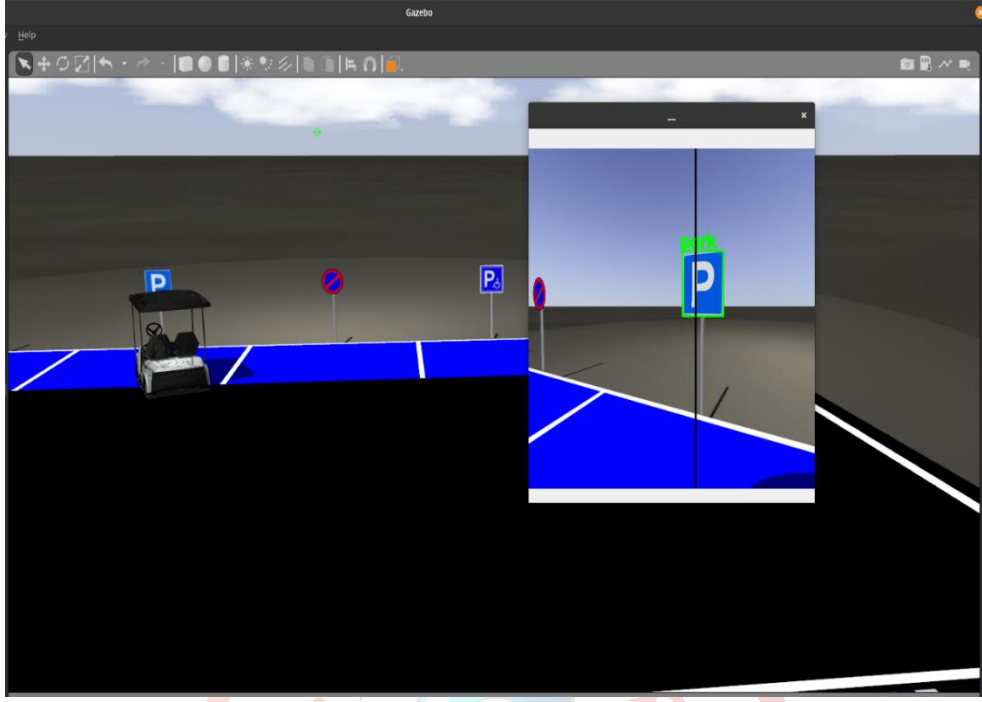
Aracın yönelim açısı ve anlık hedef arasındaki fark değeri kullanarak düzlem hatası hesaplanır. Bu hata değeri sıfırlanana kadar devam edecek olan bir PID kontrolü, arka planda çalışmaktadır.

Düzlem hatası hesaplama işlemi, aracın park tabelasına olan uzaklığının önceden belirlenen metre cinsinde olan minimum parka giriş değerine eşit olana kadar devam etmektedir.

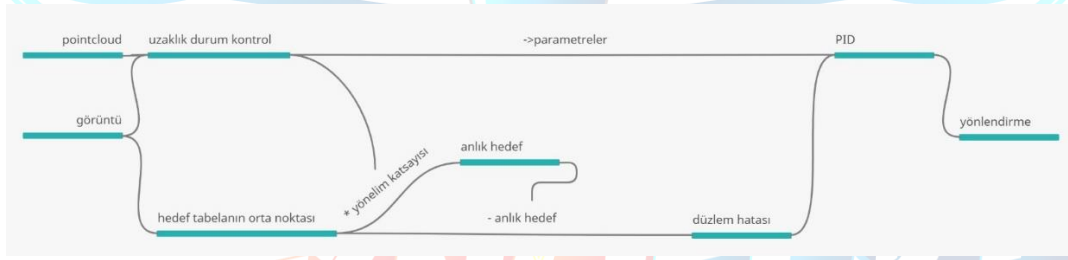
Minimum parka giriş değerine ulaşıldıktan sonra yönelim katsayısı değiştirilerek anlık hedef tabelaya göre güncellenmektedir. Bu güncellenmenin yardımıyla araç şeritlerin üzerinden geçmeden uygun park alanına park edecektir. Park algoritması, çalışacağı ortam ve çevre faktörleri yarışma şartnamesinde belirtilenler dikkate alınarak kodlanmıştır. Park algoritmasının işleyişi Şekil 15 ve Şekil 16'da verilmiştir. Park algoritmasının akışı ise Şekil 17'de verilmiştir.



Şekil 15: Uygun park tabelasına oransal yaklaşım



Şekil 16: Park alanına yaklaşım sonucu park için son manevra.



Şekil 17: Park algoritmasının akışı

6.7. Optimum Tekerlek Açısı Algoritması

Şerit takibiyle otonom sürüş sağlanırken aracın şeritlere uygun olarak hareket etmesi için ve yalpalama sorunu olmadan araç tekerlek açısını sönmölemek için optimum tekerlek açısı algoritması geliştirilmiştir. Bu algoritmada araç bilgisayarının hızlı çalışması ön planda tutularak iki değer arası normalizasyon işlemi uygulanmıştır. Uygulanan normalizasyon işlemi Denklem 3'teki gibi formüle edilmiştir.

$$x^n = (b - a) * \frac{x - \min_x}{\max_x - \min_x} + a \quad (3)$$

x^n : Normalizasyon işlemi üretilen tekerlek açısı

b: maksimum tekerlek açısı a: minimum tekerlek açısı x: anlık şeride uzaklık

$\min_x$: istenen minimum şeride uzaklık $\max_x$: istenen maksimum şeride uzaklık

Oluşturulmuş olunan ve yukarıda belirtilen formülün şerit takip algoritmasında kullanımını bir örnek üzerinde incelenecek olursa:

Tekerlek açısının orta noktası 127,5 olarak baz alınmış olup 0 değeri tam solu temsil ederken 255 değeri tam sağı temsil etmektedir. Şeride en fazla uzaklık 3,5 metre, en yakın uzaklık ise

1.5 metre olarak seçilmiştir ve anlık şeride uzaklığın 2.5 metre olduğunu varsayalım. O halde bu değerleri kullanarak formül değerlendirilirse Denklem 4'te sonuç oluşacaktır.

$$x^n = (255 - 0) * \frac{2.5 - 1.5}{3.5 - 1.5} + 0 \quad (4)$$

İşlem sonucunda x^n değeri yani normalizasyon işlemiyle üretilen tekerlek açısı 127,5 olarak sonuç üretmektedir. İşlem sonucundan da görüldüğü üzere 3,5 metre en fazla uzaklık ve 1,5 metre en az uzaklık değerleri arasında 2,5 metrelik bir uzaklıkta olması durumunda aracın 127,5 değerini tekerlek açısı olarak ürettiği gözlemlenmiştir ve aracın düz bir şekilde ilerleyeceği belirlenmiştir. Bu algoritma sayesinde dinamik şekilde tekerlek açısı üretilebilmektedir.

6.8. Sinyal Algoritması

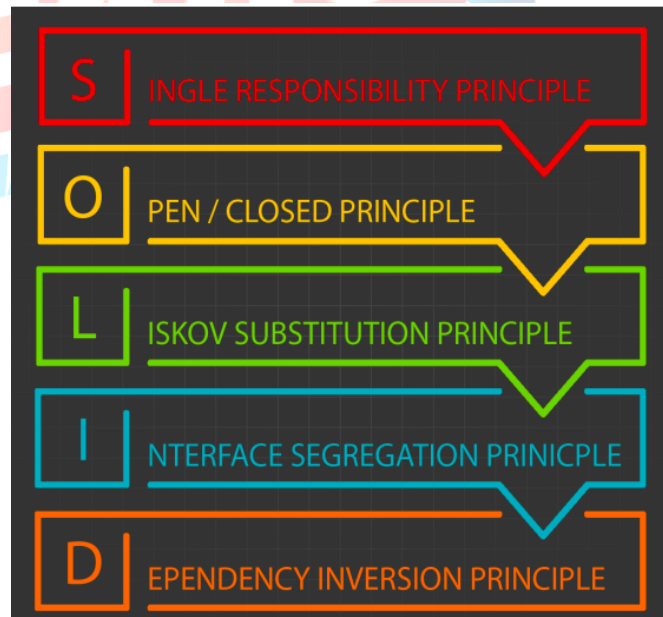
Sinyal algoritması, şerit değiştirme, dönüşler, durak alanına giriş ve çıkışlarda kullanılmak üzere tasarlanmıştır. Şerit değiştirme durumunda geçilecek olan şeridin müsaitliği kontrol edilir ve müsait olması durumunda öncelikle CAN Bus haberleşme modülü kullanılarak araca sinyal komutu gönderilmektedir. Şerit değiştirme işlemi gerçekleştikten sonra tekrardan CAN Bus haberleşme modülüne sinyal işleminin sonlandırılması için komut gönderilmektedir. Dönüşlerde ise derin öğrenme modeliyle levha algılandıktan sonra dönülecek olan yönün müsaitliği kontrol edilmektedir ve dönüş yapılabilme şartı sağlanıyorsa sinyal verme işlemi gerçekleştirilir.

7. Güvenlik Önlemleri

Bu kısımda otonom sürüş algoritmaları gerçekleştirilirken oluşabilecek risk durumlarına karşı alınmış güvenlik önlemlerinden bahsedilmektedir.

7.1. SOLID İlkeleri

Bu kısımda yazılımın güvenli bir şekilde çalışması için geliştirilen algoritma SOLID prensipleri dikkate alınarak oluşturulmuştur. SOLID yazılım prensipleri, nesne tabanlı yazılım için geliştirilen 5 önemli tasarım ilkesinin genel adıdır [14]. Her harf bir ilkeyi belirtir. Bu ilkeler Şekil 18'de gösterilmiştir.



Şekil 18: SOLID prensipleri

SOLID prensiplerinin kullanılma amaçları:

- Geliştirilen algoritmanın gelecekteki durumlara uyum sağlayabilmesi,
- Algoritmaya yeni bir özellik ekleneceği zaman kodun üzerinde yapılan değişikliklerin en aza indirgenmesi,
- Kod üzerinde bir hata oluştuğu zaman daha az zaman kaybıyla kodun düzenlenmesi,
- Son olarak algoritma üzerinde anlam karmaşıklığının en aza indirilmesidir.

Bu ilkelerin özelliklerini kısaca açıklamak gerekirse:

Single Responsibility Principle: Bir sınıfa yalnızca bir görev verilmelidir, yani bir sınıfın sadece bir işi yapması gerekir.

Open/Closed Principle: Bir sınıf yeni bir özellik kazanırken var olan özelliklerini koruyabilir olmalıdır.

Liskov Substitution Principle: Geliştirilen algoritma üzerinde bir değişiklik yapmadan alt sınıflar, üst sınıfların yani türetildikleri sınıfların yerine kullanılabilir olmalıdır.

Interface Segregation Principle: Verilen görevleri, tek bir arayüz üzerinde toplamak yerine birden fazla arayüz üzerinde toplanmalıdır.

Dependency Inversion Principle: Sınıflar arasındaki bağımlılıklar minimum seviyeye indirgenmelidir. Özellikle üst seviye sınıflar alt seviye sınıflara bağlı olmamalıdır.

Geliştirilen otonom sürüş algoritmaları, levha tespit algoritmaları ve şerit tespit algoritmaları anlatılan SOLID prensiplerine uygun olarak geliştirilmiştir. Bu sayede geliştirilen algoritmalar çalışması daha güvenli ve stabil olurken koda müdahale etmek daha kolay bir hal almıştır.

7.2. LIDAR Sensörü ile Alınan Güvenlik Önlemleri

Aracın seyri esnasında karşısına çıkabilecek çeşitli nesneler ve yayalar, sürüş güvenliğini tehdit edebilecek başlıca unsurlardandır. Bu gibi durumlardan sakınmak için LIDAR' dan gelen verilerden faydalanılacaktır. Bu yüzden LIDAR sensöründen gelen verilerin doğru bir şekilde gelmesi güvenlik açısından büyük önem arz etmektedir. Geliştirilen algoritmada LIDAR sensöründen gelen veriler 180 derece ve aracın önünü kapsayacak şekilde filtrelenmiştir. Bu sayede aracın arkasını veren LIDAR verileri kullanılmamıştır. Ayrıca LIDAR sensöründen gelen verilerin anlamlandırılmasında araç bölgelerinden faydalanılmıştır. Doğru bölgeden doğru veriyi alabilmek için LIDAR sensörünün çeşitli bölgelerine engeller koyularak bölgeler kesin olarak belirlenmiştir. Bu sayede yanlış bir bölgeden veri gelerek aracın yanlış şekilde hareket etmesi önlenmiştir. Son olarak LIDAR sensörü ile aracın önüne engel çıkması durumunda engele olan uzaklık LIDAR sensöründen alınıp elde edilen uzaklık verisi ve aracın mevcut hızı kullanılarak hesaplanan güvenli uzaklık değerinden daha az olursa araç önce yavaşlayacak daha sonra duracaktır. Araç yavaşlarken engelin yoldan çıkması durumunda araç normal seyrine devam edecektir. Araç yoluna çıkan engelin araca olan uzaklığı aracın anlık hızı kullanılarak hesaplanan ani fren eşik değerinden daha düşükse ani fren protokolü devreye girecek ve araç ani fren yaparak duracaktır.

7.3. Şeritten Çıkma Durumu

Araç kamerasından gelen filtrelenmiş şeritlerin herhangi birine aşırı yaklaşma veya şeritten çıkma durumu tespit edildiğinde aracın saptığı yaw açısının tersine bir direksiyon açısı verilip ani bir manevra gerçekleştirilmesini sağlayarak aracın şeritten çıkma ve kaza durumlarının engellenmesi planlanmıştır.

7.4. Nesne Tespiti Önlemleri

Yapay zekâ ile tespit edilen levhalar için model bir doğruluk oranı tahmin etmektedir. Ayrıca aracın nesnelere olan uzaklığı tespit edilerek karar algoritması için yüksek doğruluk oranı hedeflenmektedir. Bu doğruluk oranının %80'den aşağı olması durumunda otonom sürüş için

rota belirlemede nesne tespit algoritmasının düşük öncelikli olarak kullanılması amaçlanmıştır. Eğer aracın nesneye olan uzaklığı karar algoritmasının uygulanacağı mesafe ve doğruluk oranı halen %80'nin altında ise araç yavaşlatılarak nesne tespitinde yaşanan gürültülerin engellenmesi amaçlanarak kontrol algoritmasının doğruluğunun desteklenmesi amaçlanmıştır.

7.5. Araç Hareket Halindeyken Gelen Sensör Verilerinin Uzun Süreli Kesilmesi

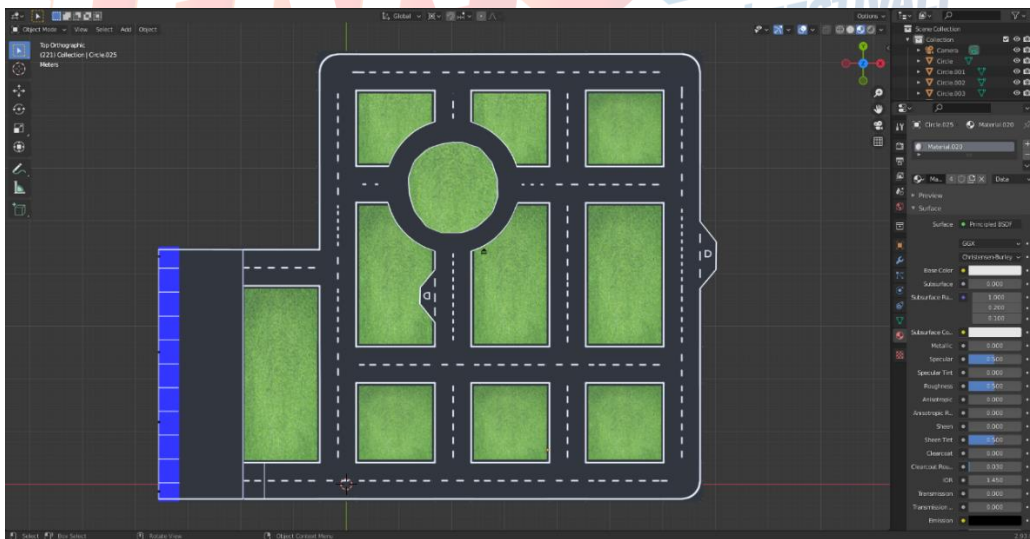
Araç hareket komutu aldıktan sonra gelen sensör verilerine göre hareket sağlamaktadır. Bu sensör verileri üzerinden gelen verinin kesilmesi durumunda son aldığı veriye göre hareket edeceğinden dolayı aracın kaza yapma ihtimali artarak tehlikeli bir durum oluşturmaktadır. Bu sorunun önüne geçmek için belirli aralıklarla algoritma yardımıyla sensörlerden gelen verilerin durumu kontrol edilmektedir. Bir aksilik olması durumunda gelen verilerde araca durma komutu verilerek bu sorunun önüne geçilmektedir.

7.6. Araç Üzerinde Bulunan Güvenlik Önlemleri

TEKNOFEST komitesi tarafından verilen araçta iki adet acil durum butonu bulunmaktadır. Birinci buton aracın direksiyonunun yanında yer alırken, ikinci buton ise araç bilgisayarının olduğu yerde aracın arkasında bulunmaktadır. Acil bir durum oluşması durumunda direksiyon yanındaki veya araç arkasındaki buton kullanılarak aracın haberleşme ve elektrik hattı güvenli bir şekilde kesilebilmektedir.

8. Simülasyon

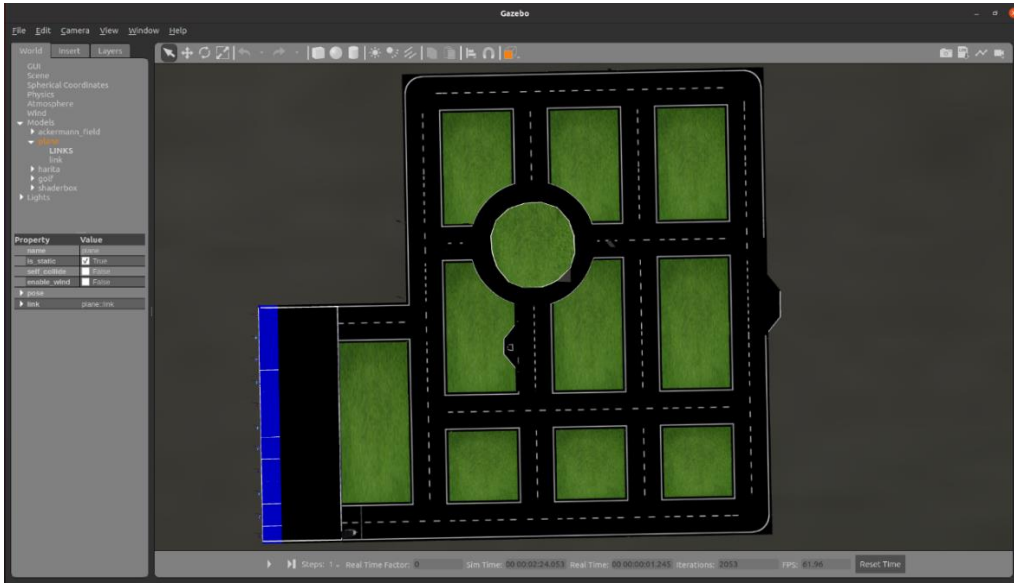
Teknik şartnamede istenilen Gazebo simülasyon ortamı üzerinde çalışmalar yapılmıştır [15]. TEKNOFEST tarafından verilen hazır araç modeli simülasyon ortamına entegre edilmiştir. Yapılan simülasyon çalışmaları, Ubuntu 20.04 üzerinde gerçekleştirilmiştir. Araç ile haberleşmenin sağlanması ve algoritmaların geliştirilebilmesi için ROS mimarisi seçilmiştir. Ubuntu 20.04 üzerinde doğru çalışan ROS Noetic kurulumu yapılmıştır. Simülasyon ortamında araç ile bağlantı sağlandıktan sonra şartnameye uygun haritanın tasarlanmasına başlanmıştır. Harita tasarımı, Blender programı yardımıyla üç boyutlu olacak şekilde yapılmıştır. Haritanın tasarımı tamamlandıktan sonra levha tasarımına geçilmiştir. Parkurda olacak olan levhalar haritada uygun yerlere yerleştirilmiş olup tamamlanan haritanın COLLADA formatında çıktısı alınmıştır. Tasarlanan haritanın görüntüsü Şekil 19'da verilmiştir.



Şekil 19: Şartnameye Uygun Tasarlanan Haritanın Blender Üzerinde Görüntüsü

Gazebo'da model entegrasyonu COLLADA formatıyla yapılmaktadır. Blender programında tasarlanan harita, COLLADA formatına çıkartılmış ve Gazebo simülasyon ortamına

entegrasyonu gerçekleştirilmiştir. Entegrasyon gerçekleştirildikten sonra haritanın testleri yapılmıştır. Test sırasında bulunan hatalar giderilmiş ve harita tasarımı sonuçlandırılmıştır. Gazebo simülasyon ortamına entegre edilen haritanın görüntüsü Şekil 20’de verilmiştir.

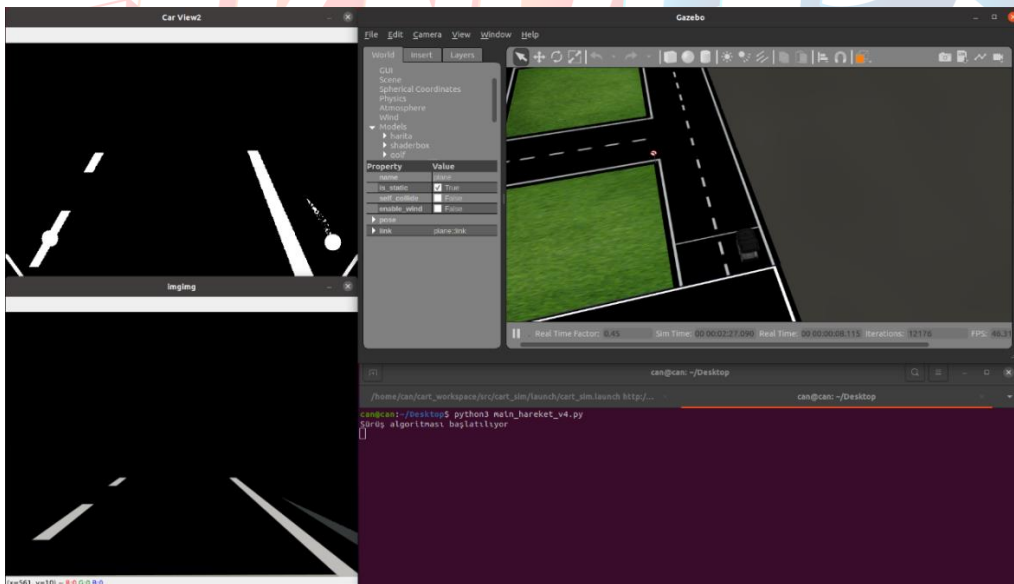


Şekil 20: Tasarlanan Haritanın Gazebo Ortamı Üzerinde Görüntüsü

8.1. Yarışma Senaryosu İçin Simülasyon Sonuçları

Geliştirilen otonom sürüş algoritmalarıyla yapılan görevlerin çıktıları bu kısımda yer almaktadır.

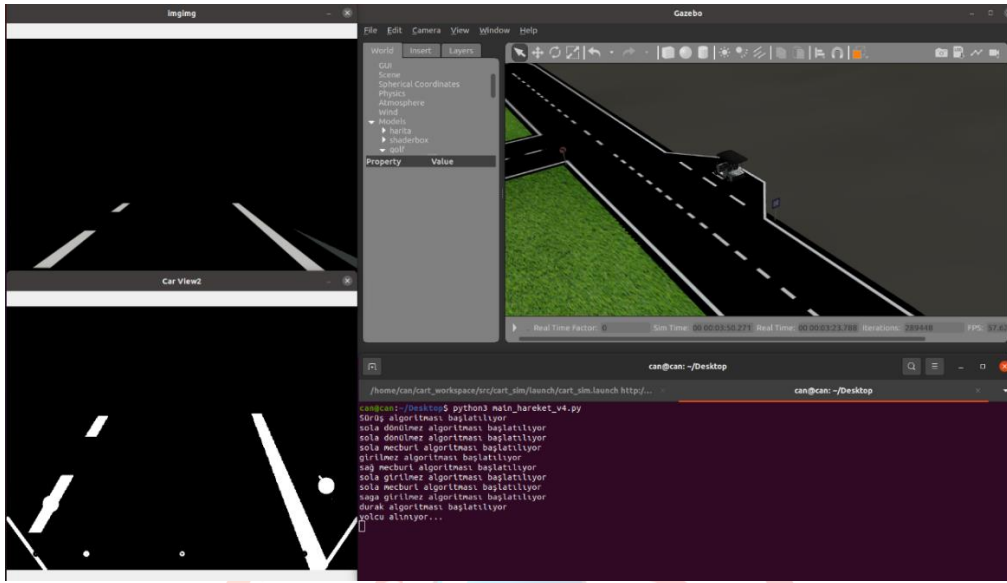
İlk görev olarak aracın harekete başlayıp başlangıç çizgisini geçmesi gerekmektedir. Şekil 21’de aracın harekete geçtiğini gösterir görsel verilmiştir. Bu görevin başarılı olmasıyla araç, 200 puan almıştır.



Şekil 21: Aracın Harekete Geçip Başlangıç Konumundan Çıkmaya Başlaması

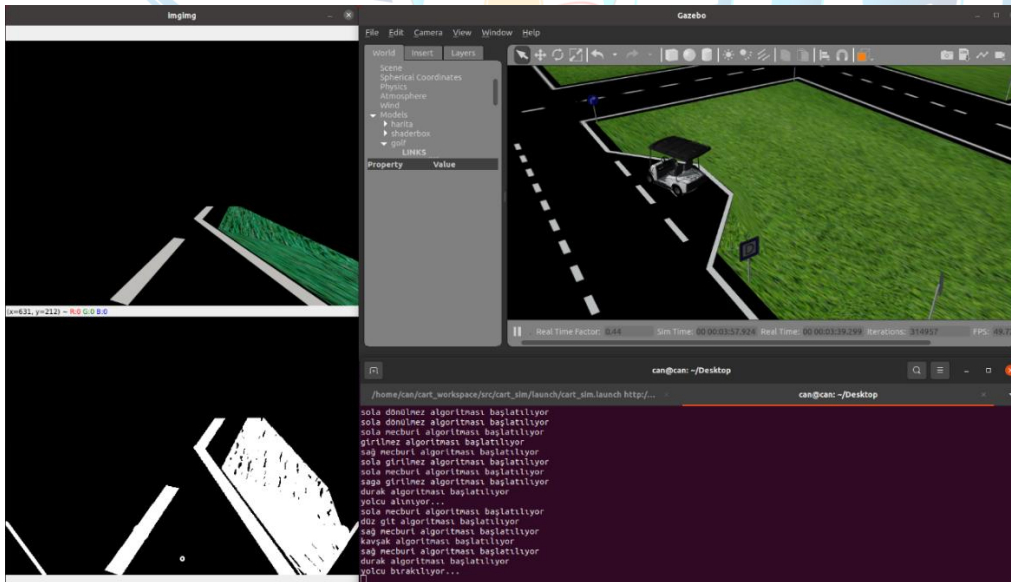
İkinci görevde aracın durağa uygun şekilde girip yolcu alması beklenmektedir. Araç, durağa girdikten 30 saniye sonra duraktan çıkacaktır. Durak tespiti için levha tespitinden gelen veriler incelenmekte ve mesafe hesaplaması yapılmaktadır. Yolcu alma görevinin başarıyla

gerçekleştirildiği Şekil 22’de gösterilmiştir. Bu görevin başarılı olmasıyla araç, 500 puan almıştır.



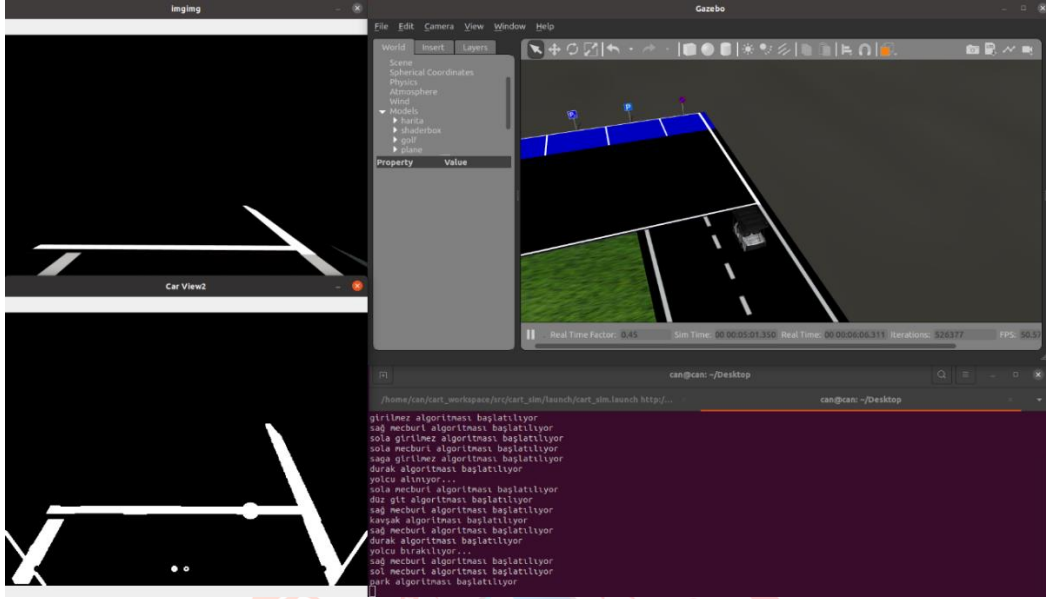
Şekil 22: Aracın Birinci Duraktan Yolcu Alması

Üçüncü görevde ise aracın aldığı yolcuları ikinci durağa bırakması beklenmektedir. İkinci görevde çalıştırılan durak tespit ve mesafe hesaplama algoritmaları bu görevde de çalıştırılmaktadır. Gerekli kontroller yapılarak durağa girilmekte ve yolcu bırakılmaktadır. Yolcu bırakma görevinin başarıyla gerçekleştirildiği Şekil 23’te gösterilmiştir. Bu görevin başarılı olmasıyla araç, 500 puan almıştır.

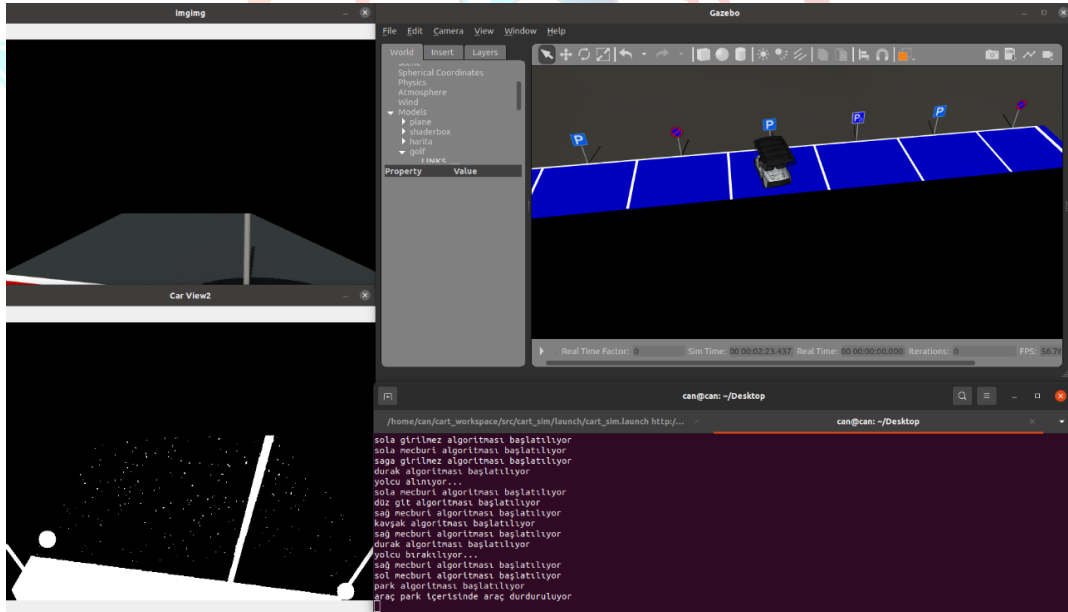


Şekil 23: Aracın İkinci Durağa Yolcu Bırakması

Dördüncü ve beşinci görevlerde ise aracın park alanına ulaşması ve uygun park alanına park etmesi beklenmektedir. Bu görevleri başarıyla tamamlayabilmek için çeşitli otonom sürüş algoritmalarından yararlanılmıştır. Park alanına giriş yapıldığında boş ve park edilebilir alanlar tespit edilerek uygun park alanına giriş sağlanmıştır. Park alanına ulaşma ve uygun park alanına parkın gerçekleştirildiği Şekil 24 ve Şekil 25’te gösterilmiştir. Araç, park alanına ulaşarak 500 puan; uygun ve boş bir park alanına park ederek 500 puan almıştır.



Şekil 24: Aracın Parkuru Bitirip Park Alanına Ulaşması



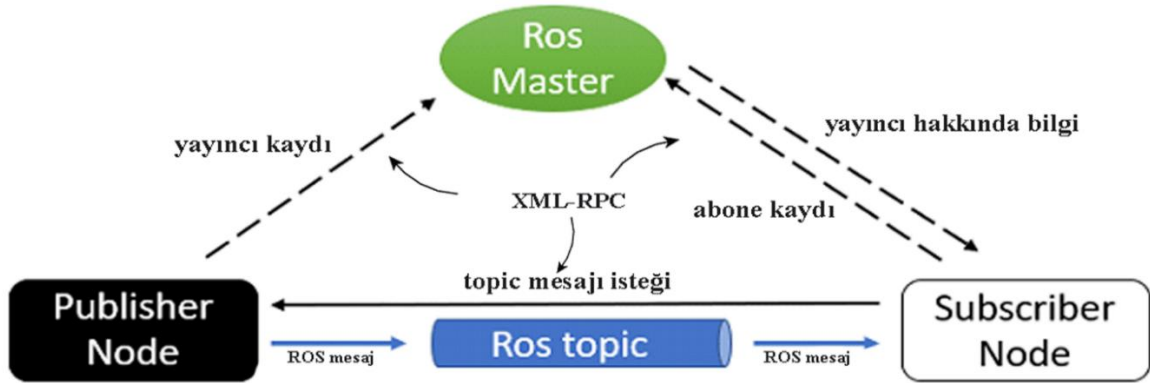
Şekil 25: Aracın Uygun ve Boş Bir Park Alanına Park Etmesi

9. Sistem Entegrasyonu

LIDAR ve ZED kamera sensörleri NVIDIA Jetson AGX Xavier geliştirme kartına bağlanmıştır. Bu geliştirme kartı araç bilgisayarı olarak kullanılmaktadır. Diğer sensörler ve donanımlar ise CAN Bus aracılığıyla araç bilgisayarına bilgi sağlayacaktır. Araç bilgisayarı Ubuntu 18.04 işletim sistemine sahip olacak olup işletim sistemi üzerinde otonom sürüş algoritmaları için gerekli olan işlemler ROS (Robot Operating System) mimarisinden faydalanılarak Python programlama diliyle yazılmıştır. ROS mimarisi Kısım 9.1'de anlatılmıştır. Sisteme entegre edilen bir diğer kısım ise nesne tespit mimarisidir. Nesne tespit mimarisine ait bilgiler Kısım 9.2'de verilmiştir. Tüm bu geliştirilen algoritmaların ve sensörlerin sisteme entegrasyonu sonucunda araç hareketi CAN Bus'a komut göndererek sağlanmaktadır.

9.1. ROS Mimarisi

ROS (Robot Operating System), robotlar için açık kaynak bir meta-işletim sistemidir. Bu sistem donanım soyutlama, düşük seviye cihaz kontrolü, sık kullanılan fonksiyonların kolay dâhil edilmesi, işlemler arasında mesaj aktarımı ve paket yönetimi gibi olanaklar sunmaktadır. ROS bu bakımdan diğer alternatif robotik uygulamalarına benzemektedir. ROS'u diğer muadillerinden ayıran en temel özelliği parçalı yürütülebilir ve bölümlenebilir bir tasarım sunmasıdır [16]. ROS'un bir diğer faydası da önceden geliştirilmiş hazır algoritmaları yükle ve kullan formatında hızlı bir şekilde çalıştırabilmesidir. Ayrıca algoritmada performans karşılaştırması, gözetim yapabilme kolaylığı sağlaması ve hata ayıklama mekanizmasına sahip olmasıdır. İçerisinde ROS barındıran araçtaki sensörlerle ses ve görüntü gibi dış dünyadan toplanan veriler bilgisayara ulaşır. Bilgisayarda algoritma ve kullanım alanına göre işlenen veri, alıcı konumundaki robota komut olarak geri dönebilmektedir. Buradaki bilgisayar ve robotun iletişimi ROS arayüzünde bulunan mesaj ve topic olarak adlandırılan öğeler ile yapılabilmektedir [17]. ROS sistemi publisher-subscriber (yayıncı-abone) arasında topic üzerinden iletişim sağlayan düğümlerin meydana getirdiği bir mimaridir. Düğüm denilen yapı işlem gerçekleştiren kod dizinleridir. Her biri farklı bir işlem yürüten bu düğümlerin senkronize bir şekilde birbirleriyle iletişim kurması gerekmektedir. Bu bağlantıyı gerçek zamanlı ve senkronize bir şekilde sağlamak için bir sunucuya ihtiyaç vardır. Bu sunucuya ROS Master ismi verilir. Düğümler ROS Master'a, publish (yayın) ve subscribe (abone) bilgileriyle kaydolurlar. Şekil 26'da gösterildiği gibi XMLRPC tabanlı olan Master sunucusunun kullanılması ile birbirleriyle iletişime geçen düğümler sayesinde istenilen işlem gerçekleştirilmiş olur.



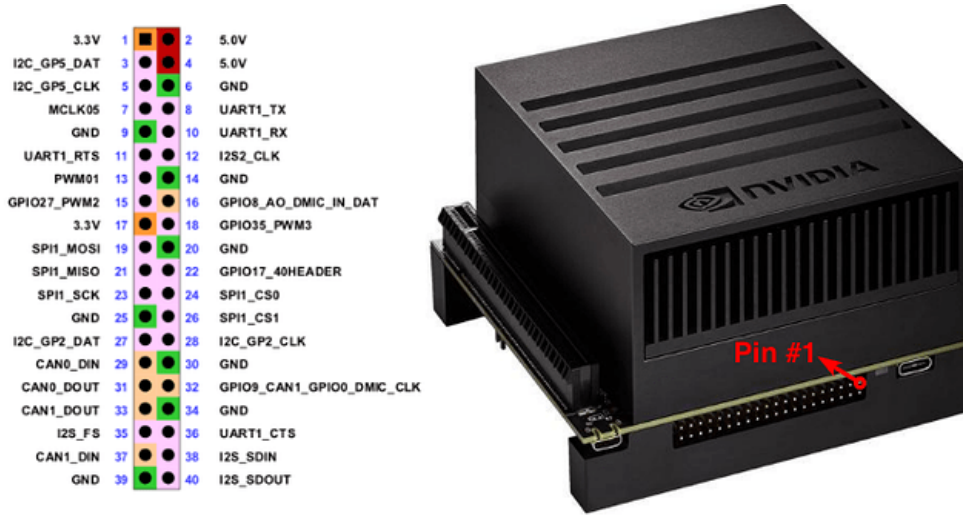
Şekil 26: ROS'un temel çalışma prensibi

9.2. Nesne Tespiti Mimarisi

Geliştirilen nesne tespit algoritmaları araç bilgisayara entegrasyonu sağlanmıştır. PyTorch kütüphanesi ve YOLOv5 nesne tespit algoritması sisteme entegre edilmiştir. Nesne tespitinden gelen verileri otonom sürüş algoritmasında kullanabilmek için ROS aracılığıyla sonuçlar publish (yayınlanma) edilmektedir. Bu sayede YOLOv5 ve ROS kullanarak kameradan gelen görüntünün işlenmesiyle trafik levhalarının tespiti sağlanmış ve sistem entegrasyonu sağlanmıştır. Bu entegrasyon adımının ardından yapay zekâ destekli otonom sürüş gerçekleştirilmiştir.

9.3. CAN Bus Haberleşme Kurulumu

Araçın otonom kontrolünü sağlarken gaz, fren, direksiyon açısı ve sinyal vermek için araç bilgisayarı üzerinden CAN Bus ile haberleşmesi sağlanır. Bunun için Python kullanılmaktadır ve haberleşmesinin sağlanması için Python-CAN kütüphanesi kurulmalıdır. Araç bilgisayarı üzerinde CAN0 veya CAN1 pinlerine CAN Bus bağlanarak haberleşmesi sağlanır. Araç bilgisayarı üzerindeki haberleşme pinleri Şekil 27'de verilmiştir.



Şekil 27: Araç Bilgisayarı Üzerindeki Haberleşme Pinleri

9.4. ZED Kamera Entegrasyonu

ZED 2 kamera, araçla USB portu üzerinden haberleşmektedir. ZED 2 kamerayı aktifleştirmek için önce ZED 2 kamera, araç bilgisayarının USB portuna takılır. Daha sonra, Stereolabs firmasının internet sitesi üzerinden ZED SDK indirilir ve kurulum gerçekleştirilir. ZED SDK kurulumu gerçekleştirildikten sonra ZED Explorer çalıştırılarak ZED 2 kameranın çalıştığı doğrulanır. ZED 2 kameranın çalıştığı doğrulandıktan sonra ROS bağlantısını sağlamak için “zed-ros-wrapper” paketi indirilir ve kurulur. Daha sonra “zed-ros-wrapper” paketi çalıştırılır ve ZED 2 kameranın, ROS içerisinde çalıştığı doğrulanır.

9.5. LIDAR Sensörü Entegrasyonu

Velodyne Puck LIDAR sensörü, Ethernet portu üzerinden çalışmaktadır. Velodyne Puck LIDAR, araç bilgisayarının Ethernet portuna takılır. LIDAR çalıştırıldıktan sonra, araç bilgisayarından sabit IP ataması yapılır. IP ataması yapıldıktan sonra, Velodyne Puck LIDAR'ı ROS içerisinde kullanabilmek için gerekli olan “velodyne” paketi kurulur. “velodyne” paketi çalıştırılır ve LIDAR'ın çalışıp çalışmadığı Python'da ROS kodu yazılarak kontrol edilir.

9.6. IMU ve GPS Sensör Entegrasyonları

CAN Bus üzerinden gelen SBG IMU ve GPS verileri Python-CAN kütüphanesi aracılığıyla Python kodunda anlamlandırılarak veriler okunup otonom sürüş algoritmalarında kullanılmaktadır.

10. Test ve Doğrulama

Levha tespitinde kullanılacak olan YOLOv5 modelinin hem simülasyon ortamında hem de gerçek ortamda testleri yapılmıştır. Simülasyon ortamında yapılan testlerde amaçlanan, trafik işaretlerinin doğru bir şekilde tespit edilmesidir. Gerçek ortamda yapılan testlerde ise hedeflenen, trafik işaretlerinin doğru bir şekilde tespit edilmesi ve tespit oranlarının her bir trafik işareti için 0,8 üzerinde olmasıdır. Simülasyon ortamında ve gerçek ortamda yapılan testlerin çıktıları Şekil 28 ve Şekil 29'da verilmiştir.



Şekil 28: Simülasyon Ortamında Yapılan Levha Tespit Testi Sonucu



Şekil 29: Gerçek Ortamda Yapılan Levha Tespit Testi Sonucu

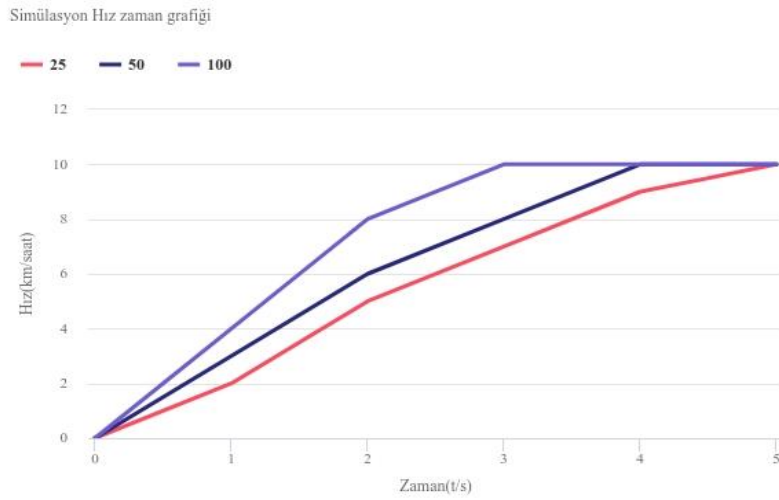
Yapılan levha tespit testleri sonucunda, aşağıdaki sonuçlara ulaşılmıştır:

- Simülasyon ortamı için hazırlanan tespit modelinin simülasyon ortamında doğru sonuç verdiği,
- Simülasyon ortamı için hazırlanan tespit modelinin gerçek ortamda doğru sonuç vermediği ve tespit oranlarının %50'nin altında olduğu,
- Gerçek ortamda kullanılmak üzere, TTVS temel alınarak hazırlanan levha tespit modelinin parkurda doğru sonuç verdiği fakat tespit oranlarının düşük olduğu,
- TTVS temelli levha tespit modelinin, parkurdan alınan verilerle güçlendirilerek hazırlanan geliştirilmiş levha tespit modelinin parkurda doğru sonuç verdiği ve tespit oranlarının yüksek olduğu gözlemlenmiştir.

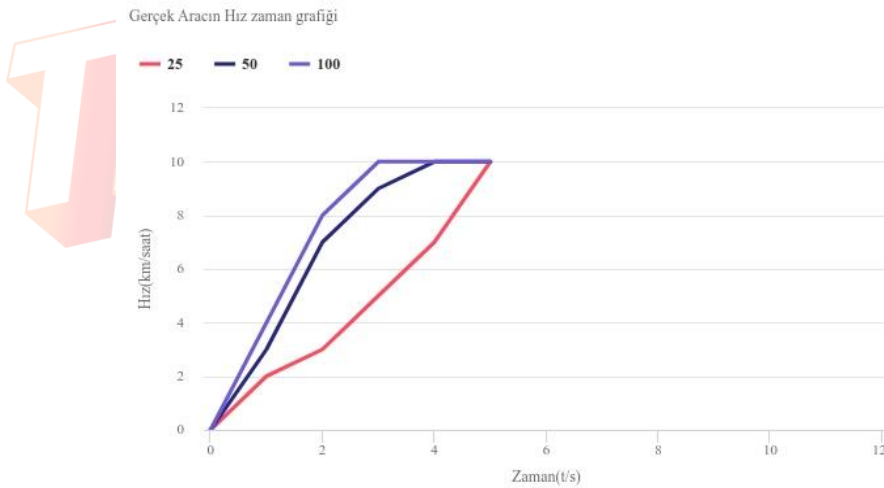
Hazır aracın hareketini daha doğru sağlamak amacıyla simülasyon ortamında ve gerçek ortamda bazı testler uygulanmıştır. Uygulanan testler şunlardır:

- İvmelenme testi
- Frenleme testi
- Dönüş testi

İvmelenme testinde aracın 10 km/s hıza kaç saniyede ulaştığı, çeşitli gaz değerleriyle ölçülmüştür. Yapılan ivmelenme testi sonucunda, simülasyon ortamında aracın gaz tepkilerine daha geç yanıt verdiği gözlemlenmiştir. İvmelenme testinin sonuçları Şekil 30 ve Şekil 31’de verilmiştir.

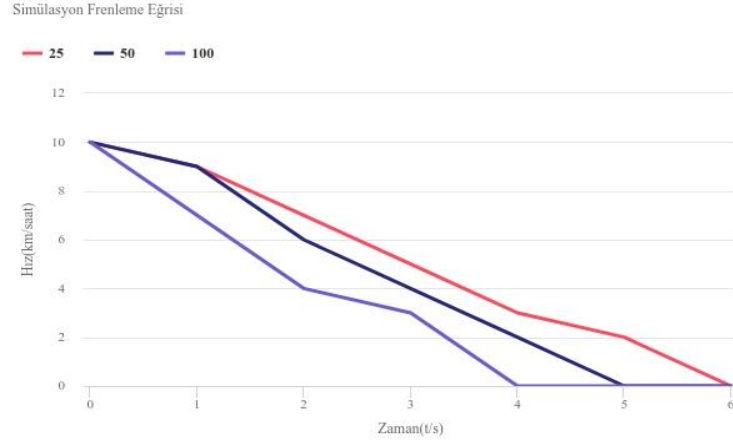


Şekil 30: Simülasyon Ortamında Yapılan İvmelenme Testi Sonucu

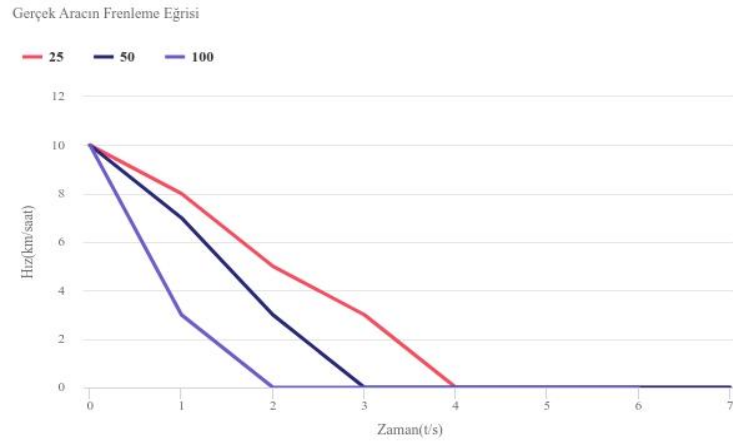


Şekil 31: Gerçek Ortamda Yapılan İvmelenme Testi Sonucu

Frenleme testinde aracın 10 km/s hızdan 0 km/s hıza kaç saniyede düştüğü, çeşitli fren değerleriyle ölçülmüştür. İvmelenme testinde de olduğu gibi simülasyon ortamında araç, gaz tepkilerine gerçek ortama göre daha geç tepki vermektedir. Frenleme testinin sonuçları Şekil 32 ve Şekil 33’te verilmiştir.

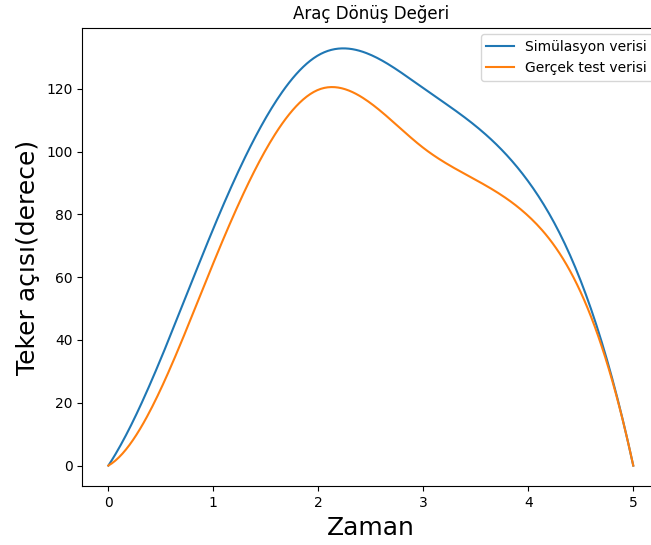


Şekil 32: Simülasyon Ortamında Yapılan Frenleme Testi Sonucu



Şekil 33: Gerçek Ortamda Yapılan Frenleme Testi Sonucu

Dönüş testinde ise aracın kaç saniyede maksimum dönüş açısı değerine ulaştığı ölçülmüştür. Yapılan dönüş testi sonucunda simülasyon ortamında aracın dönüş yeteneğinin daha iyi olduğu gözlemlenmiştir. Simülasyon ortamında aracın mekanik durumu mükemmel olduğundan simülasyon ortamında mekanik problem kaynaklı bir dönüş yeteneği kaybı olmayacaktır. Gerçek ortamdaki araç ise aktif olarak kullanıldığından mekanik yıpranmaya uğrayacak ve dönüş yeteneğinde kayıp olacaktır. Dönüş testinin sonucu Şekil 34’te verilmiştir.



Şekil 34: Simülasyon Ortamında ve Gerçek Ortamda Yapılan Dönüş Testleri Sonucu

Yarışma kapsamında RACLAB Sigun takımı tarafından geliştirilen otonom sürüş algoritmalarının hazır araç üzerinde çalışmasını gösterir araç test videosuna <https://www.youtube.com/watch?v=2xA85oD8-Lo> adresinden ulaşılabilir.

11. Referanslar

- [1] TTVS (Türkiye Trafik Veri Seti)
- [2] Velodyne Puck Hi-Res LIDAR
- [3] ZED2 Stereo Kamera
- [4] SBG Ellipse microII IMU
- [5] NVIDIA Jetson AGX Xavier Geliştirme Kartı
- [6] CAN Protokolü
- [7] cdn.t3kys.com/media/upload/userFormUpload/yYWJ3iHOYHDu1Nnm5LTwRaxIWXFofJue.pdf (2021, Robotaksi Binek Otonom Araç Yarışması, RACLAB Sigun Takımı KTR)
- [8] <https://analizsimulasyon.com/arac-dinamigi-kamber-kaster-ve-toe-acilari/>
- [9] Burha, M. (Haziran 2010). İki Akstan Dümenlenen Üç Akslı Özel Maksatlı Bir Taşıtın Direksiyon Mekanizmasının Kinematik Tasarımı. İstanbul: Yüksek Lisans Tezi.
- [10] <https://datagenetics.com/blog/december12016/index.html>
- [11] kgm.gov.tr/SiteCollectionDocuments/KGMdocuments/Trafik/IsaretlerElKitabi/TrafikIsaretleriElKitabi2015.pdf
- [12] cdn.t3kys.com/media/upload/userFormUpload/Kkz4mPep7znqcfgKqDU3NWWWEZCEOEpsF.pdf (2022, Robotaksi Binek Otonom Araç Yarışması, RACLAB Sigun Takımı ÖTR)
- [13] <https://github.com/ultralytics/yolov5>
- [14] medium.com/bilişim-hareketi/solid-nedir-ne-değildir-12c8bdfeda1c
- [15] <https://gazebo.org/home>
- [16] <http://wiki.ROS.org/ROS/Introduction>
- [17] <https://devnot.com/2020/ros-robot-operating-system-nedir/>